

DIVE DEEPER, BRANCH LATER: SELF-EVOLVING EVALUATORS ACROSS OPEN-ENDED TASKS

Anonymous authors

Paper under double-blind review

Working manuscript: quantitative displays use simulated data; empirical evaluation pending.

ABSTRACT

Evaluating open-ended outputs requires task-dependent judgments across multiple aspects of quality that must be balanced in context. Existing proxy metrics and model-based evaluators remain imperfect on such judgments and do not consistently align with human preferences. Humans can often tell which of two concrete outputs is better, but it is much harder to turn that ability into a comprehensive and effective evaluation procedure. Evaluator self-evolution offers a way to learn such procedures from human feedback, but open-ended evaluation makes the evolution direction difficult to determine. The same evaluation error can suggest several plausible improvements, while a useful direction may require multiple revisions before producing a measurable gain. An unsuccessful early attempt therefore provides limited evidence about whether the direction is ineffective or simply underdeveloped. In the default model-driven evolution setup we tested, this ambiguity often shifted subsequent revisions toward alternative directions. We present IterEval, a framework for self-evolving evaluators across open-ended tasks. IterEval combines disagreement-based preference mining with depth-first evaluator evolution, explicitly maintaining an improvement direction across revisions and continuing it until diagnostic results support moving on or its search budget is exhausted. Unsuccessful candidates can guide further implementation without replacing the best evaluator found so far. Across diverse open-ended tasks, IterEval produces more sustained improvement under matched search budgets and yields evaluators that better reflect human preferences. The evolved evaluators also improve downstream output selection and critic-guided refinement, while their scores provide useful training rewards on representative generation tasks.

1 INTRODUCTION

Evaluating open-ended tasks requires case-specific judgments across multiple dimensions of quality, with no single evaluation standard sufficient for every case (Ye et al., 2024; Kim et al., 2024b). A summary may need to preserve fine-grained details when completeness matters, yet emphasize brevity and the central argument when rapid understanding is the goal. Likewise, visual richness may be valuable for a promotional webpage but detrimental to a task-oriented interface. Metrics and model-based evaluators capture useful aspects of quality, yet do not consistently match human preferences (Fabbri et al., 2021; Xu et al., 2023; Zheng et al., 2023; Hu et al., 2024). Humans themselves, however, often cannot fully express their judgments as a comprehensive, executable evaluation procedure (Shankar et al., 2024b). Human preferences are therefore better treated as iterative feedback for improving an evaluator.

Existing approaches use human feedback to train evaluator models (Xu et al., 2023), learn or refine evaluation criteria (Liu et al., 2024; Lv et al., 2026), and select evaluation functions (Shankar et al., 2024b). Prompt and skill optimizers offer a complementary mechanism for revising instructions from execution feedback (Agrawal et al., 2026; Yang et al., 2026). We present IterEval, a framework for self-evolving evaluators across open-ended tasks. IterEval directly revises an explicit evaluation procedure while building on the base model’s existing semantic and reasoning capabilities. This

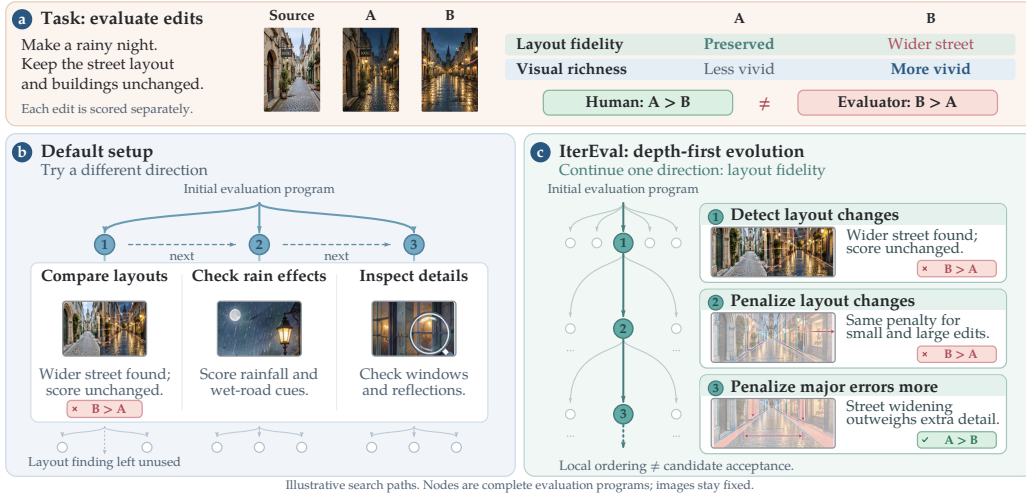


Figure 1: Dive deeper, branch later. Panel (a) contrasts A’s preserved layout with B’s greater visual richness and illustrates opposite human and evaluator rankings. Panel (b) branches across plausible corrections; dashed arrows show attempt order. Panel (c) develops one correction through detection, scoring, and severity weighting. Nodes denote complete evaluation programs; hollow nodes are unexplored alternatives. The corrected ordering is local to this example, not candidate acceptance or generalization. Images, preferences, and paths are constructed illustrations, not experimental evidence.

allows the evaluator to adapt without updating the base model parameters and makes individual changes easier to inspect and test.

Extending evaluator self-evolution to new open-ended domains, however, raises two challenges: obtaining useful human preferences and reliably turning them into improvements to the evaluation procedure.

Obtaining preference data that is useful for evolution is itself nontrivial: under a limited human feedback budget, the choice of comparisons can affect learning efficiency (Muldrew et al., 2024; Oh et al., 2025). Hand-designed comparisons tend to reflect the cases and failure modes anticipated by the designer, while broad collections can contain many easy or redundant comparisons that reveal little about where the evaluator still falls short. We therefore use disagreement-based preference mining to prioritize comparisons on which existing evaluation signals disagree. When preference labels already exist, preference mining selects informative examples from the available data; otherwise, it prioritizes comparisons for new human annotation.

Preference mining selects the comparisons used for evaluator evolution, but does not specify how the evaluator should be revised. Pairwise preferences identify the better output without specifying the fine-grained criteria behind that choice (Ye et al., 2024; Lv et al., 2026). The same evaluation error revealed by human preferences can therefore suggest several plausible improvement directions, while limited human feedback often provides little basis for choosing among them. This ambiguity is especially important in open-ended evaluation, where a useful direction may require several related revisions before producing a measurable gain. An unsuccessful early candidate evaluator therefore leaves open whether its direction is ineffective or simply underdeveloped.

This uncertainty also affects how evaluator evolution proceeds. In the default evaluator-evolution setup we tested, each new revision was proposed by the model from previous candidates and their outcomes. We observed a tendency for unsuccessful early candidates to be followed by revisions in alternative directions rather than deeper development of the current one. Over repeated iterations, evolution can therefore spread effort across multiple plausible directions before any one of them has been developed far enough to judge reliably.

IterEval uses depth-first evaluator evolution to explicitly maintain an improvement direction across revisions (Figure 1c; Figure 2b). The search continues developing the current direction after unsuc-

successful intermediate candidates, using their diagnostic results to guide the next revision. The best evaluator is maintained separately and replaced only by an improved candidate. Evolution moves to a new direction when diagnostic results provide sufficient evidence against the current one or when its search budget is exhausted.

We evaluate IterEval across structured visual artifacts, images and 3D, text generation, and automated research, covering both reused preference datasets and newly collected human preferences. Disagreement-based preference mining identifies preference subsets that lead to stronger evaluator improvement than standard sampling from the same data source. Under matched search budgets, depth-first evaluator evolution continues developing directions after unsuccessful early candidates and produces more sustained improvement than the default setup. The resulting evaluators also achieve stronger agreement with held-out human preferences and improve downstream output selection and critic-guided refinement. On representative tasks, their scores further provide useful training rewards for improving generation quality.

Our contributions are as follows:

- We introduce IterEval, a framework that learns from human preferences and revises the full evaluation procedure across heterogeneous open-ended task domains.
- Depth-first evaluator evolution explicitly maintains an improvement direction across revisions, preventing an unsuccessful intermediate candidate from automatically ending development of that direction. Disagreement-based preference mining selects informative preferences from existing labeled data or prioritizes comparisons for new human annotation.
- We provide a cross-domain evaluation of preference mining, evaluator evolution, final agreement with human preferences, and downstream utility under controlled annotation and search budgets.

2 RELATED WORK

Learning evaluators from human feedback. Human feedback supports both training evaluator models and constructing explicit evaluation rules. Improving these judgments matters beyond assessment: when evaluators guide generator optimization, reward hacking can exploit their errors rather than improve human-perceived quality (Wang et al., 2026). Learned reward models and trained judges adapt model weights to produce scores or critiques aligned with human judgments (Xu et al., 2023; Kim et al., 2024a; Ke et al., 2024). Other approaches use feedback to generate, select, or refine explicit evaluation criteria, including preference-grounded query-specific rubrics (Shankar et al., 2024a; Liu et al., 2024; Shankar et al., 2024b; Lv et al., 2026). Agent-as-a-Judge and WebGrader extend evaluation through tool-based inspection and evolving verifier skills, respectively (Zhuge et al., 2025; Chen et al., 2026a). IterEval builds on this move toward explicit evaluation programs: with the base model fixed, it uses human feedback to revise both how evidence is gathered and how that evidence determines judgments.

Program optimization and agent self-evolution. Automated optimization treats prompts, modules, or agent code as editable state, allowing adaptation without changing the underlying model weights. DSPy and TextGrad optimize components of predefined programs (Khattab et al., 2024; Yuksekgonul et al., 2025), while GEPA and SkillOpt use execution feedback and model-generated reflection to revise textual state (Agrawal et al., 2026; Yang et al., 2026). Agent-level search expands the editable structure and preserves alternatives through archives of designs or programs in ADAS and DGM, distinct harnesses in HarnessBank, and a persistent revision tree in SkillHEX (Hu et al., 2025; Zhang et al., 2026; Luo et al., 2026; Feng et al., 2026). SkillLens distinguishes extracting a skill from successfully using it, motivating attention to executed behavior rather than proposed changes alone (Huang et al., 2026). Within this line of work, IterEval focuses on whether to continue an improvement direction after a candidate fails to improve: it maintains the direction across revisions and decides its continuation separately from candidate acceptance.

Active preference acquisition for evaluation. Active preference learning selects which comparisons receive feedback, with disagreement sampling rooted in query-by-committee (Seung et al.,

1992; Muldrew et al., 2024; Oh et al., 2025). Benchmark² addresses a related selection problem for measurement, constructing test sets for ranking consistency, discriminability, and capability alignment (Qian et al., 2026). IterEval instead selects feedback to improve an evaluator, reducing the priority of repetitive comparisons on which existing signals agree without treating agreement as correctness. It can mine existing labels or request new annotations; its endpoint is evaluator improvement under matched label-access or annotation budgets, respectively, rather than disagreement in the selected set.

3 ITEREVAL

3.1 OVERVIEW AND SETUP

Starting from a seed evaluator E_0 , IterEval revises an executable evaluation program E_θ using human feedback, with the base model M fixed. Its central distinction is between accepting a candidate and continuing an improvement direction. A direction identifies an evaluation error and an intended correction to the evaluation program; a candidate is a complete program produced by one revision. The direction is the unit of continued exploration; the complete program is the unit of selection. Revisions can change what the evaluator observes, which criteria it applies, how it routes tools, and how it combines evidence (Section 3.4).

The search maintains

$$S_t = (B_t, W_t, d_t, z_t), \quad (1)$$

where B_t is the best accepted program, W_t is the working program being revised, d_t is the active direction, and z_t retains observed outcomes, the agent’s interpretations, and unresolved questions. Each revision belongs to d_t and adds its outcomes to z_t , whether or not the candidate replaces B_t . Revisions continue within the active direction until the agent ends it or search resources are exhausted. Thus rejection can change which program is edited next without discarding the direction under investigation (Figure 2b).

For a task x , the evaluator scores each output y separately, without seeing the competing output or human preference. Within a domain D sharing an evaluation standard, the objective is agreement with human preferences on unseen inputs,

$$\max_{\theta} J_D(\theta), \quad J_D(\theta) = \Pr_{(x, y^+, y^-) \sim \mathcal{P}_D, \xi} [\bar{s}_\theta(x, y^+) > \bar{s}_\theta(x, y^-)]. \quad (2)$$

Here y^+ is the human-preferred output, $\mathcal{P}_D$ is the domain’s input-and-preference distribution, and $\bar{s}_\theta$ is the score obtained under a fixed repeated-scoring protocol with execution randomness ξ . Appendix C specifies sampling weights, repeated scoring, ties, and incomplete executions.

3.2 DISAGREEMENT-BASED PREFERENCE MINING

Preference mining supplies the human feedback used by evolution, drawing comparisons from existing datasets or newly generated outputs. Broad preference pools can contain many repetitive comparisons that expose few new evaluation errors. Disagreement-based preference mining lowers the sampling priority of comparisons on which existing evaluation signals consistently agree. Pairs with conflicting signals receive higher priority, while a random anchor checks for shared blind spots.

For selected comparisons, compatible human preferences are reused when available; otherwise, blinded annotation supplies the missing preferences. Both routes supply the same feedback interface to evolution, preserving each label’s source and its associated artifacts, signals, and execution records. Unselected labels remain hidden from evolution. For existing labels, sampling policies are compared at equal label-access budgets; for new annotation, they are compared at equal annotation counts, with annotation time reported separately. Random-anchor labels count toward the corresponding budget. Appendix B defines these units and the detailed ranking rules. Sampling efficiency is measured by subsequent evaluator improvement, not by disagreement in the selected data alone.

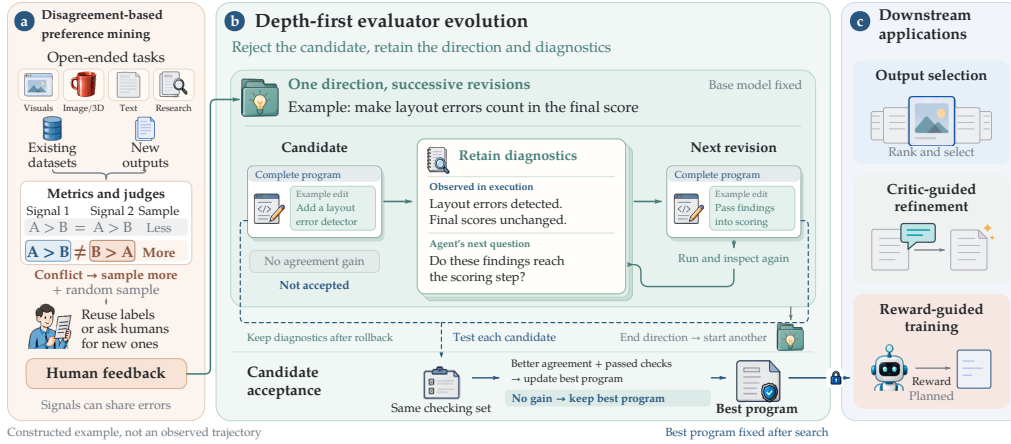


Figure 2: IterEval: iterative evaluator self-improvement from human feedback. Panel (a) mines comparisons from existing datasets or new outputs, reusing compatible preferences or collecting missing ones, with a random sample to check for shared signal errors. Panel (b) retains a rejected candidate's diagnostics to inform another revision within the same direction, separating observed outcomes from the agent's next question. Dashed links send each candidate to the same checking set for acceptance; green links carry diagnostic information, not automatic code inheritance. The agent can end the direction and start another if resources remain. Panel (c) uses the best program, fixed after search, for output selection, critic-guided refinement, and planned reward-guided training. The example and AI-generated pictograms are constructed illustrations, not observed trajectories or measured improvements.

3.3 DEPTH-FIRST EVALUATOR EVOLUTION

Each cycle either starts a direction or continues the active one using its accumulated diagnostics. The agent either revises the working program or ends the direction with a recorded explanation. A candidate is evaluated for acceptance, while its outcomes inform the next revision within the same direction. Ending the direction allows the next session to propose a new one from the best program. Algorithm 1 summarizes this ordinary revision loop.

The proposal subset $\mathcal{D}_{\text{prop}}$ exposes artifacts, human feedback, and execution traces; the disjoint checking subset $\mathcal{D}_{\text{dev}}$ returns only aggregate candidate measurements. Both are development data. The agent can inspect detailed proposal cases, while all candidates are accepted or rejected using the same checking set. Diagnostic calls, unsuccessful trials, and sessions that end a direction remain in the search cost record.

Choosing a direction. With no active direction, the agent inspects proposal cases and previous trials, identifies an evaluation error, and states the correction it intends to make. The target error and intended correction guide subsequent revisions. For example, adding a tool and then changing how its output affects the judgment can develop the same correction. The controller assigns a direction identifier and starts from $W_t = B_t$. The identifier links successive revisions to the same starting program and trial history. The agent may refine its account of the correction as diagnosis proceeds, but earlier failures remain in the record. The controller preserves this identifier and history; it does not verify that successive revisions retain the same semantic intent. Direction choice and the first revision can occur in the same editing session.

Revising and diagnosing. Within the active direction, the agent reads the working program and prior diagnostic results, requests further observations, and submits a complete candidate C_t . A submission records its edits, intent, and inspected proposal cases. Every valid candidate that changes its parent is evaluated on the same checking set; verified measurements may be reused for exact replays. Execution traces and measurements on proposal cases record tool outputs, changes in judgments, repaired errors, and regressions. The diagnostic state z_t retains these observations alongside the agent's interpretations and unresolved questions, linked to the candidate and its actual parent.

Algorithm 1 Depth-first evaluator evolution (ordinary revision loop)**Require:** Seed E_0 , proposal feedback, checking data, configured search and execution limits

```

1: Measure  $E_0$ ;  $B \leftarrow E_0$ ,  $W \leftarrow E_0$ ,  $d \leftarrow \emptyset$ ; initialize history
2: while search resources remain do
3:   if  $d = \emptyset$  then
4:      $d \leftarrow$  a direction chosen from proposal errors and history; initialize  $z$ ;  $W \leftarrow B$ 
5:   end if
6:   Investigate  $(W, d, z)$  using proposal feedback and authorized diagnostics
7:   if the agent ends  $d$  then
8:     Record its explanation, observations and costs; retain  $(d, z)$  in history
9:      $d \leftarrow \emptyset$ ; continue
10:  end if
11:  Submit candidate  $C$  within  $d$ ; record its actual parent, edits and intent
12:  if  $C$  is valid and differs from its parent then
13:    Obtain checking and local outcomes; reuse verified records for exact replays
14:    Save  $B_{\text{old}} \leftarrow B$ ; update  $B$  by Equation 3
15:    Choose  $W \in \{C, B_{\text{old}}\}$  by the conservative parent rule (Appendix E)
16:  end if
17:  Retain all outcomes and costs in  $z$  and history; keep  $d$  active
18:  Stop if execution integrity requires interruption
19: end while
20: On normal completion, return the Train-selected program after terminal assessment

```

Observed outcomes establish what the program produced, whereas interpretations explain what the agent proposes to test next.

For example, a tool may detect missing content while the final judgment ignores the finding. The agent can test this interpretation by revising how the judgment uses the tool output, rather than repeating the detection step. It likewise investigates missing observations, incorrect interpretation, or a correction that was applied but did not help; these explanations are not treated as verified facts.

Candidate acceptance and direction continuation. Candidate acceptance updates the best program according to recorded checking agreement $\hat{J}_{\text{dev}}$,

$$\begin{aligned}
 a_t &= \mathbb{I}[\hat{J}_{\text{dev}}(C_t) > \hat{J}_{\text{dev}}(B_t)] \mathbb{I}[\Gamma_t(C_t)], \\
 B_{t+1} &= \begin{cases} C_t, & a_t = 1, \\ B_t, & a_t = 0. \end{cases} \tag{3}
 \end{aligned}$$

The guard Γ_t requires valid execution and passes only when the coverage and regression checks in Appendix E are satisfied. This rule preserves the recorded best development agreement. It does not decide whether the direction receives another revision.

Within a continuing direction, a conservative parent rule chooses whether the next revision starts from C_t or the pre-update best program B_t (Appendix E). This rule uses local as well as checking comparisons to assess whether further edits should build on the candidate. If local comparisons are incomplete, even an accepted candidate may not become the next working program; search then edits B_t while retaining C_t as the new best program. Changing the parent neither reverses candidate acceptance nor erases the candidate’s diagnostic results.

Continuing before branching. While a direction is active, the next session receives its identifier, current working program, latest trial, and accumulated diagnostic results. The agent can request further diagnostics or submit another revision within that direction, regardless of whether the previous candidate was accepted. To end the direction, it must issue an explicit action, implemented as `pause`, with a recorded reason. The session instruction asks for the unresolved question and why no different informative test is ready. For example, the agent may conclude that an implemented correction does not address the target error after investigating possible implementation problems. The agent judges when to end a direction; the controller records this decision without verifying that its reason is sufficient.

In the ordinary revision loop, a new direction is proposed only after the agent ends the active direction, starting from the current best program. Neither rejecting a candidate nor returning to the best

program automatically ends the direction. Configured search and execution limits can also stop exploration, leaving the direction unresolved. On normal completion, search returns the best program selected on Train, subject to the terminal checks in Appendix E, or E_0 if no candidate qualifies. Terminal validation cannot change this choice or reopen search; independent held-out assessment follows method freeze.

3.4 REVISION SPACE

The editable program comprises artifact adapters A , rubrics and dimensions K , evaluator modules V , tools T , evidence routing Π , and fusion G . A revision may change several connected components together: adding a tool, routing its output, and changing how the final judgment uses that output. The complete program, rather than each partial edit, is executed and measured. The base model remains fixed throughout. Figure A1 and Appendix E specify these interfaces and the detailed execution rules.

4 EXPERIMENTS

We test evaluator agreement and selection (C1), refinement (C2), and reward-guided training (C3), with separate mechanism studies. Numerical displays use a simulated ideal scenario for reporting design, not observed outcomes.

Tasks and assessment. We prioritize webpages (SV4), detailed captions (IG1), summaries (TG1), and research ideas (AR1), separately adapting each evaluator. The 22-domain candidate scope is not a completed benchmark (Appendix A). The primary test uses unseen within-domain inputs, independently of mining scores; derivatives and near-duplicate sources stay together. Out-of-distribution tests are optional.

Comparisons and resources. Table 1 includes static judges, learned signal fusion, SkillOpt (Yang et al., 2026), GEPA (Agrawal et al., 2026), and Meta-Harness (Lee et al., 2026). Official optimizers retain their native history and search policies; GEPA receives a full-program interface. Adaptive methods share feedback, model access, tools, and resource caps; search cost includes reflection, diagnostics, failures, and validation. Independent searches measure variation. Text-only IterEval controls for SkillOpt’s narrower edit space (Appendix D).

4.1 CORE RESULTS

C1: evaluator agreement and output selection. Methods score the same outputs pointwise, without human labels or opponent information. Table 1 separates strict preference agreement from Best-of- N accuracy and normalized rank regret. Selection requires human-ranked groups, not pair-only data. The ideal scenario places IterEval at 78.0% agreement, 9.0 percentage points above Seed and 1.8 above GEPA, with better selection and lower regret. This pattern would support gains in both judging pairs and choosing outputs; agreement alone would not establish the latter.

The assigned gains span all four domains rather than a single favorable task (Figure 3c). IterEval’s deployment cost is 1.18 times Seed’s in panel (b), between GEPA and Meta-Harness. Actual cost measurements must determine whether this quality–cost relation holds.

C2: critic-guided refinement. A frozen generator refines the same fresh y_0 using Seed or IterEval critiques, or no critique (Madaan et al., 2023), at matched resources. The scenario favors IterEval over Seed on 56.0% of tie-adjusted comparisons across three domains, with fewer major regressions (Table 2). Comparisons with y_0 and no-critic revision distinguish useful critique from extra generation alone (Table A14). Actual revisions require new blinded judgments.

C3: reward-guided training. Matched policies train with frozen Seed or IterEval rewards and their own online rollouts using LoRA/GRPO (Hu et al., 2022; Shao et al., 2024). The assigned 56.3% preference for IterEval-trained outputs illustrates the intended downstream test, not a training finding. Reward scores alone cannot establish this benefit; new human judgments and reserved attribute checks are required. C3 awaits preflight and separate training authorization (Appendix K).

Table 1: **C1: final held-out performance.** Simulated ideal scenario; not experimental results. Pair: preference agreement (%); BoN: Best-of- N accuracy (%); Reg: normalized rank regret; coverage (%) uses requested inputs. Values equally weight four domains with $N = 4$; no confidence intervals are assigned.

Evaluator	Pair $\uparrow$	BoN $\uparrow$	Reg $\downarrow$	Coverage $\uparrow$
Fixed model judge	63.8	43.0	0.325	99.5
Seed evaluator	69.0	49.5	0.280	98.6
Learned signal fusion	71.0	52.0	0.259	98.6
SkillOpt	73.2	56.0	0.230	98.8
GEPA	76.2	59.0	0.209	98.9
Meta-Harness	76.8	60.5	0.202	98.7
IterEval	78.0	62.0	0.192	98.8
Eligible domains	4	4	4	4

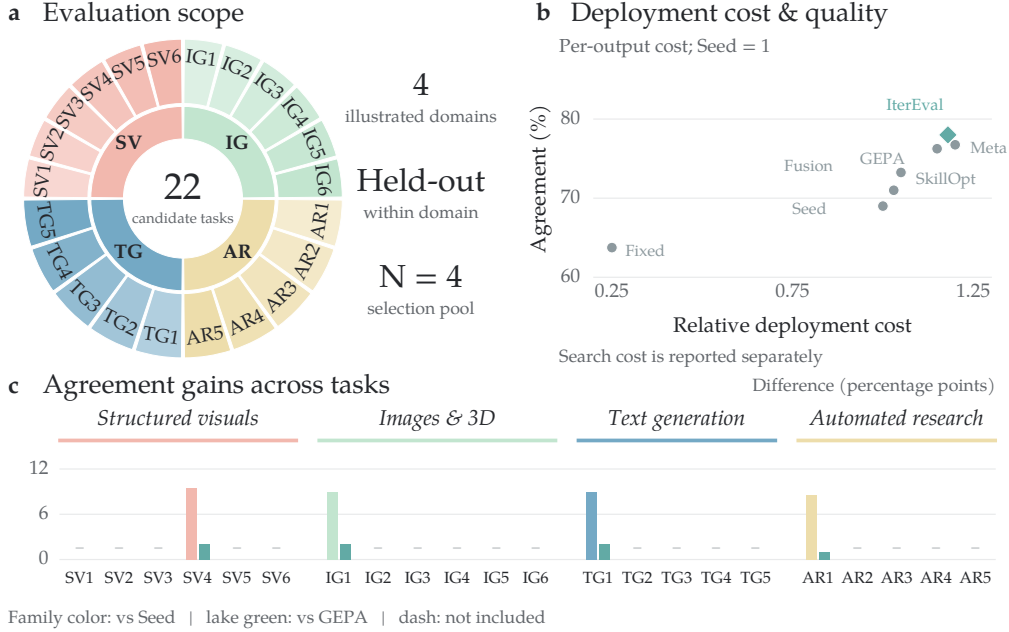


Figure 3: **C1 breadth and deployment trade-offs.** Simulated ideal scenario; not experimental results. (a) Candidate task scope, not sample shares. (b) Assigned per-output cost versus agreement. (c) Domain gains over Seed and GEPA; dashes mark domains outside this scenario. The layout reuses Table 1’s values, not independent evidence.

Table 2: **Downstream human-outcome reporting.** Simulated ideal scenario; no annotation or training was conducted. Pref.: tie-aware preference for IterEval over Seed (%). Δ Reg: major-regression difference against y_0 (C2) or Base (C3), in percentage points. Counts are assigned; C3 remains conditional.

Study	Domain	Pref. $\uparrow$	Δ Reg $\downarrow$	Judged inputs
C2	SV4	55.0	-2.5	160
C2	IG1	57.5	-3.1	160
C2	TG1	55.6	-1.9	160
C3	IG1	56.3	-2.5	160

4.2 ABLATION STUDIES

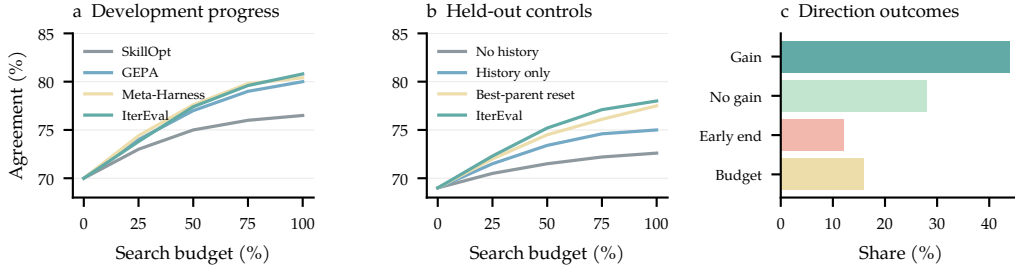


Figure 4: **Search dynamics.** Simulated ideal scenario; not observed runs. (a) Best accepted development agreement versus total search cost. (b) Independent held-out checkpoint curves for the four history/parent controls, assessed only after search. (c) Assigned direction outcomes (definitions in Appendix H). No uncertainty estimates are assigned.

Direction continuation. The four controls in Figure 4b isolate parent retention, direction priority, and history at equal cost. Their assigned contrasts are 0.5, 2.5, and 2.4 points, respectively (Table A11); independent searches must confirm this ordering.

Preference mining. Compare mined and random feedback by held-out agreement at equal revealed-label budgets, and additionally at equal human time for new annotation. The random/mined $\times$ history-only/IterEval scenario gives a 0.2-point interaction, while its time-matched comparison remains favorable after charging harder annotations (Figure A2). Both patterns require confirmation with actual label and time records.

4.3 ANALYSIS AND STUDIES

Figure A3 reserves a trace comparison; Table A13 illustrates tests of evidence use and feedback detail. Appendices C–N specify uncertainty, failures, and records for every display.

5 DISCUSSION

What would support deeper search? Later gains over history-matched search and better held-out selection would support retaining directions independently of candidate acceptance. Longer directions alone are insufficient; the parent-reset control separates direction priority from retaining intermediate implementations.

Scope and limitations. Separately adapted, within-domain evaluators do not establish universal transfer. The controller retains direction identifiers and diagnostics but cannot verify semantic continuity or sufficient reasons to end; trace audits must separate observations from agent interpretation. Agreement also need not improve downstream quality, and reward optimization can amplify evaluator errors (Wang et al., 2026). Fresh human judgments and reserved attribute checks remain necessary; the scenario establishes no empirical benefit.

6 CONCLUSION

IterEval learns an evaluation program from human feedback through disagreement-based preference mining and depth-first evaluator evolution. It separates acceptance of a complete candidate from continuation of its improvement direction. Held-out agreement, sustained search, and downstream quality jointly test whether this approach uses human feedback effectively; the ideal scenario is not empirical evidence.

AI USE STATEMENT

Generative AI assisted manuscript drafting, literature organization, and the preparation of editable LaTeX conceptual diagrams and plotting code. Figure 1 uses AI-generated illustrative images from an earlier figure-design pass, with editable vector labels and search paths. Its preferences and intermediate outcomes are constructed, not observed annotations or optimization traces. Figure 2 reuses AI-generated pictograms with editable vector labels and connectors; its example is constructed rather than an observed trajectory. Current quantitative displays use an author-requested simulated ideal scenario of aggregate values, without generated raw observations or statistical inference. Historical simulated figures remain in the repository as disclosed visual references, outside the active results and appendix. Neither templates nor simulations are observed cases, human annotations, model executions, or performance measurements. No empirical results or parameter-training runs were produced for this draft. Bibliographic identities were checked against primary metadata sources. Author review, responsibility for the final content, and the final disclosure of research-stage AI use remain to be completed before submission.

ETHICS STATEMENT

Human preferences can encode annotator and model biases rather than a universal definition of quality. Domain-specific criteria and uncertainty should remain visible, especially for automated research judgments. Any new annotation study must document consent, compensation, privacy protection, and the applicable institutional review determination before data collection. Artifact licenses and source permissions must be checked independently of technical accessibility. Generated code and evidence tools require restricted execution and controlled external access. Reward-guided optimization requires checks for exploitation of the evaluator and degradation outside its favored criteria. No private artifacts, identifiable annotation records, or experimental traces are included in this working manuscript. Study-specific approvals and release decisions are pending.

REPRODUCIBILITY STATEMENT

Sections 3.1–4 define semantic mappings, feedback, the evaluation program, evolution, and controlled evaluation. Appendices B–C specify annotation, access boundaries, and measurement. Appendix E retains the native implementation and diagnostic limits. Appendices D–N specify baseline controls, core reporting, conditional training, ablations, analyses, and the experiment manifest. Table A18 lists every active display and its required inputs. Appendix N identifies the active ideal scenario, empty empirical-record schemas, and historical simulated resources, none of which constitute empirical results. The current manuscript contains no empirical results to reproduce. Before submission, the authors must supply a versioned, appropriately anonymized implementation and permitted data-processing and analysis artifacts with completed manifests. Their availability is not asserted here.

REFERENCES

- Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziem, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alex Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. In *International Conference on Learning Representations*, volume 2026, pp. 8479–8565, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/0e9e708b6f48e14fd0ac29e167413f76-Abstract-Conference.html.
- Boshui Chen, Huiping Liu, and Shaolei Zhang. WebGrader: Training LLMs for web development with self-evolving programmatic grader. *arXiv preprint arXiv:2608.06474*, 2026a. URL <https://arxiv.org/abs/2608.06474>.
- Xin-Sheng Chen, Jiayu Zhu, Pei lin Li, Hanzheng Wang, Shuojin Yang, and Meng-Hao Guo. PresentBench: A Fine-Grained Rubric-Based Benchmark for Slide Generation. *arXiv preprint arXiv:2603.07244*, 2026b. URL <https://arxiv.org/abs/2603.07244>.

- Alexander R. Fabbri, Wojciech Kryściński, Bryan McCann, Caiming Xiong, Richard Socher, and Dragomir Radev. SummEval: Re-evaluating Summarization Evaluation. *Transactions of the Association for Computational Linguistics*, 9:391–409, 2021. doi: 10.1162/tacl_a_00373.
- Yuru Feng, Yaoqi Chen, Beidi Zhao, Qianxi Zhang, Xinjiang Wang, Jianan Lu, Zhirui Wang, Shusen Xu, Zengzhong Li, and Qi Chen. SkillHEX: Improving agent skills via hypothesis-driven autonomous exploration and exploitation. *arXiv preprint arXiv:2608.05628*, 2026. URL <https://arxiv.org/abs/2608.05628>.
- Dhruba Ghosh, Hannaneh Hajishirzi, and Ludwig Schmidt. GenEval: An object-focused framework for evaluating text-to-image alignment. In *Advances in Neural Information Processing Systems*, volume 36, pp. 52132–52152, 2023. doi: 10.52202/075280-2270.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=nZeVKeeFYf9>.
- Shengran Hu, Cong Lu, and Jeff Clune. Automated Design of Agentic Systems. In *International Conference on Learning Representations*, volume 2025, pp. 21344–21377, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/36b7acf6f6010652b3f2a433774a66fe-Abstract-Conference.html.
- Xinyu Hu, Mingqi Gao, Sen Hu, Yang Zhang, Yicheng Chen, Teng Xu, and Xiaojun Wan. Are LLM-based evaluators confusing NLG quality criteria? In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 9530–9570. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.516. URL <https://aclanthology.org/2024.acl-long.516/>.
- Zisu Huang, Jingwen Xu, Yifan Yang, Ziyang Gong, Qihao Yang, Muzhao Tian, Xiaohua Wang, Changze Lv, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Xue Yang, Dongdong Chen, Xiaoqing Zheng, and Chong Luo. From Raw Experience to Skill Consumption: A Systematic Study of Model-Generated Agent Skills. *arXiv preprint arXiv:2605.23899*, 2026. URL <https://arxiv.org/abs/2605.23899>.
- Pei Ke, Bosi Wen, Andrew Feng, Xiao Liu, Xuanyu Lei, Jiale Cheng, Shengyuan Wang, Aohan Zeng, Yuxiao Dong, Hongning Wang, Jie Tang, and Minlie Huang. CritiqueLLM: Towards an informative critique generation model for evaluation of large language model generation. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 13034–13054. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.704. URL <https://aclanthology.org/2024.acl-long.704/>.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. In *International Conference on Learning Representations*, volume 2024, pp. 54928–54958, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/file/f1cf02ce09757f57c3b93c0db83181e0-Paper-Conference.pdf.
- Seungone Kim, Jamin Shin, Yejin Cho, Joel Jang, Shayne Longpre, Hwaran Lee, Sangdoo Yun, Seongjin Shin, Sungdong Kim, James Thorne, and Minjoon Seo. Prometheus: Inducing Fine-grained Evaluation Capability in Language Models. In *International Conference on Learning Representations*, volume 2024, pp. 29927–29962, 2024a. URL https://proceedings.iclr.cc/paper_files/paper/2024/file/803485352e61e3ebf41221e4776c9fd4-Paper-Conference.pdf.
- Tae Soo Kim, Yoonjoo Lee, Jamin Shin, Young-Ho Kim, and Juho Kim. EvalLM: Interactive evaluation of large language model prompts on user-defined criteria. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*, pp. 1–21. ACM, 2024b. doi: 10.1145/3613904.3642216. URL <https://doi.org/10.1145/3613904.3642216>.

- Yuval Kirstain, Adam Polyak, Uriel Singer, Shahbuland Matiana, Joe Penna, and Omer Levy. Pick-a-Pic: An Open Dataset of User Preferences for Text-to-Image Generation. In *Advances in Neural Information Processing Systems*, volume 36, pp. 36652–36663, 2023. doi: 10.52202/075280-1594.
- Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-Harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*, 2026. URL <https://arxiv.org/abs/2603.28052>.
- Yuxuan Liu, Tianchi Yang, Shaohan Huang, Zihan Zhang, Haizhen Huang, Furu Wei, Weiwei Deng, Feng Sun, and Qi Zhang. Calibrating LLM-based evaluator. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pp. 2638–2656. ELRA and ICCL, 2024. doi: 10.63317/3posqwchnb6g. URL <https://aclanthology.org/2024.lrec-main.237/>.
- Xiaotian Luo, Dizhan Xue, Fengxingyu Wang, Chuanrui Hu, and Yafeng Deng. HarnessBank: Semantic gene-bank search with gated verification for agent-harness self-evolution. *arXiv preprint arXiv:2607.13683*, 2026. URL <https://arxiv.org/abs/2607.13683v2>.
- Changze Lv, Jie Zhou, Wentao Zhao, Jingwen Xu, Shihan Dou, Zisu Huang, Muzhao Tian, Xiaohua Wang, Zhengkang Guo, Yang Liu, Pluto Zhou, Tao Gui, Le Tian, Xiao Zhou, Xiaoqing Zheng, Xuanjing Huang, and Jie Zhou. Learning Query-Specific Rubrics from Human Preferences for DeepResearch Report Generation. *arXiv preprint arXiv:2602.03619*, 2026. URL <https://arxiv.org/abs/2602.03619>.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, volume 36, pp. 46534–46594, 2023. doi: 10.52202/075280-2019.
- William Muldrew, Peter Hayes, Mingtian Zhang, and David Barber. Active preference learning for large language models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 36577–36590. PMLR, 2024. URL <https://proceedings.mlr.press/v235/muldrew24a.html>.
- Minhyeon Oh, Seungjoon Lee, and Jungseul Ok. Comparison-based active preference learning for multi-dimensional personalization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 33145–33166. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.1590. URL <https://aclanthology.org/2025.acl-long.1590/>.
- Yasumasa Onoe, Sunayana Rane, Zachary Berger, Yonatan Bitton, Jaemin Cho, Roopal Garg, Alexander Ku, Zarana Parekh, Jordi Pont-Tuset, Garrett Tanzer, Su Wang, and Jason Baldridge. DOCCI: Descriptions of connected and contrasting images. In *Computer Vision – ECCV 2024*, pp. 291–309. Springer Nature Switzerland, 2025. doi: 10.1007/978-3-031-73027-6_17. URL https://doi.org/10.1007/978-3-031-73027-6_17.
- Qi Qian, Chengsong Huang, Jingwen Xu, Changze Lv, Muling Wu, Wenhao Liu, Xiaohua Wang, Zhenghua Wang, Zisu Huang, Muzhao Tian, Jianhan Xu, Kun Hu, He-Da Wang, Yao Hu, Xuanjing Huang, and Xiaoqing Zheng. Benchmark²: Systematic Evaluation of LLM Benchmarks. *arXiv preprint arXiv:2601.03986*, 2026. URL <https://arxiv.org/abs/2601.03986>.
- H. S. Seung, M. Oppen, and H. Sompolinsky. Query by committee. In *Proceedings of the Fifth Annual Workshop on Computational Learning Theory*, pp. 287–294. ACM, 1992. doi: 10.1145/130385.130417. URL <https://doi.org/10.1145/130385.130417>.
- Shreya Shankar, Haotian Li, Parth Asawa, Madelon Hulsebos, Yiming Lin, J. D. Zamfirescu-Pereira, Harrison Chase, Will Fu-Hinthorn, Aditya G. Parameswaran, and Eugene Wu. spade: Synthesizing data quality assertions for large language model pipelines. *Proceedings of the VLDB Endowment*, 17(12):4173–4186, 2024a. doi: 10.14778/3685800.3685835. URL <https://www.vldb.org/pvldb/vol17/p4173-shankar.pdf>.

- Shreya Shankar, J.D. Zamfirescu-Pereira, Bjoern Hartmann, Aditya Parameswaran, and Ian Arawjo. Who Validates the Validators? Aligning LLM-Assisted Evaluation of LLM Outputs with Human Preferences. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*, pp. 1–14, 2024b. doi: 10.1145/3654777.3676450.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. URL <https://arxiv.org/abs/2402.03300>.
- Chenglei Si, Diyi Yang, and Tatsunori Hashimoto. Can LLMs generate novel research ideas? a large-scale human study with 100+ NLP researchers. In *International Conference on Learning Representations*, volume 2025, pp. 94003–94092, 2025a. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/ea94957d81b1c1caf87ef5319fa6b467-Abstract-Conference.html.
- Chenglei Si, Yanzhe Zhang, Ryan Li, Zhengyuan Yang, Ruibo Liu, and Diyi Yang. Design2Code: Benchmarking Multimodal Code Generation for Automated Front-End Engineering. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 3956–3974, 2025b. doi: 10.18653/v1/2025.naacl-long.199.
- Xiaohua Wang, Muzhao Tian, Yuqi Zeng, Zisu Huang, Jiakang Yuan, Bowen Chen, Jingwen Xu, Mingbo Zhou, Wenhao Liu, Muling Wu, Zhengkang Guo, Qi Qian, Yifei Wang, Feiran Zhang, Ruicheng Yin, Shihan Dou, Changze Lv, Tao Chen, Kaitao Song, Xu Tan, Tao Gui, Xiaoqing Zheng, and Xuanjing Huang. Reward Hacking in the Era of Large Models: Mechanisms, Emergent Misalignment, Challenges. *arXiv preprint arXiv:2604.13602*, 2026. URL <https://arxiv.org/abs/2604.13602>.
- Jiazheng Xu, Xiao Liu, Yuchen Wu, Yuxuan Tong, Qinkai Li, Ming Ding, Jie Tang, and Yuxiao Dong. ImageReward: Learning and Evaluating Human Preferences for Text-to-Image Generation. In *Advances in Neural Information Processing Systems*, volume 36, pp. 15903–15935, 2023. doi: 10.52202/075280-0700.
- Yifan Yang, Ziyang Gong, Weiquan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Yuqing Yang, Dongdong Chen, Xue Yang, and Chong Luo. SkillOpt: Executive strategy for self-evolving agent skills. *arXiv preprint arXiv:2605.23904*, 2026. URL <https://arxiv.org/abs/2605.23904v2>.
- Seonghyeon Ye, Doyoung Kim, Sungdong Kim, Hyeonbin Hwang, Seungone Kim, Yongrae Jo, James Thorne, Juho Kim, and Minjoon Seo. FLASK: Fine-grained language model evaluation based on alignment skill sets. In *International Conference on Learning Representations*, volume 2024, pp. 55361–55414, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/f41b4a6b202adcd8e150a9d4f124d8f6-Abstract-Conference.html.
- Mert Yuksekogunul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. Optimizing generative AI by backpropagating language model feedback. *Nature*, 639(8055):609–616, 2025. doi: 10.1038/s41586-025-08661-4.
- Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents. In *International Conference on Learning Representations*, volume 2026, pp. 104223–104294, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/aa5f5e6eb6f613ec412f1d948dfa21a5-Abstract-Conference.html.
- Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, XiongHui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. AFlow: Automating Agentic Workflow Generation. In *International Conference on Learning Representations*, volume 2025, pp. 34040–34077, 2025. URL <https://openreview.net/forum?id=z5uVAKwmjf>.

- Hao Zheng, Xinyan Guan, Hao Kong, Wenkai Zhang, Jia Zheng, Weixiang Zhou, Hongyu Lin, Yaojie Lu, Xianpei Han, and Le Sun. PPTAgent: Generating and Evaluating Presentations Beyond Text-to-Slides. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 14402–14418, 2025. doi: 10.18653/v1/2025.emnlp-main.728.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, Hao Zhang, Joseph Gonzalez, and Ion Stoica. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, volume 36, pp. 46595–46623, 2023. doi: 10.52202/075280-2020.
- Mingchen Zhuge, Changsheng Zhao, Dylan R. Ashley, Wenyi Wang, Dmitrii Khizbullin, Yongyang Xiong, Zechun Liu, Ernie Chang, Raghuraman Krishnamoorthi, Yuandong Tian, Yangyang Shi, Vikas Chandra, and Jürgen Schmidhuber. Agent-as-a-judge: Evaluate agents with agents. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 80569–80611. PMLR, 2025. URL <https://proceedings.mlr.press/v267/zhuge25a.html>.

A DOMAINS, DATA SOURCES, AND STUDY READINESS

A *domain* is a collection of tasks sharing core quality dimensions, their priorities, the overall decision rule, and the human annotation protocol. The experimental unit is a separately adapted evaluator within one such standard. Table A1 lists 22 candidate domains in four families; inclusion does not establish experimental readiness. The initial core candidates are TD-SV4, TD-IG1, TD-TG1, and TD-AR1. They are priorities, not completed experiments or a commitment to fill missing domains with proxy labels. A domain enters the final comparison only after its data, standard, executable interface, and independent test are frozen. All admitted domains must be reported regardless of their results.

Table A1: Complete domain panorama. The four families organize the scope; every row requires its own standard and readiness audit. These are study candidates, not 22 completed evaluations.

ID	Domain	Core quality considerations
Structured visual artifacts		
TD-SV1	Brief-conditioned single-slide generation	Brief coverage, hierarchy, readability, composition, and visual consistency.
TD-SV2	Screenshot-conditioned webpage reconstruction	Reference content, spatial arrangement, typography, and visual fidelity.
TD-SV3	Data-to-visualization generation	Data fidelity, valid encodings, readability, and communicative usefulness.
TD-SV4	Specification-conditioned webpage generation	Requirement satisfaction, functional behavior, usability, and visual organization.
TD-SV5	Source-grounded slide-deck generation	Source faithfulness, narrative, coverage, slide readability, and deck consistency.
TD-SV6	Topic-conditioned slide-deck generation	Relevance, factual support, narrative completeness, audience fit, and presentation quality.
Image generation and understanding		
TD-IG1	Detailed image-caption generation	Grounding, coverage, relation accuracy, specificity, and absence of hallucination.
TD-IG2	Text-to-image generation	Prompt alignment, object and relation accuracy, visual coherence, and preference.
TD-IG3	Single-source instruction-based image editing	Instruction fulfillment, preservation of unrequested content, and image quality.
TD-IG4	LR-conditioned image super-resolution	Reference fidelity when available, perceptual detail, artifact avoidance, and plausible texture.
TD-IG5	Text-to-3D object generation	Prompt correspondence, geometry, appearance, and multi-view consistency.
TD-IG6	Image-to-3D object generation	Source-view fidelity, plausible unseen geometry, appearance, and multi-view consistency.
Text generation		
TD-TG1	Source-grounded document summarization	Faithfulness, relevance, coverage, coherence, and concision.
TD-TG2	RAG-supported long-form answer generation	Answer correctness, evidence grounding, attribution, completeness, and organization.
TD-TG3	Machine translation	Meaning preservation, omissions and additions, fluency, terminology, and style.
TD-TG4	Instruction-constraint-following response generation	Constraint satisfaction, substantive correctness, relevance, and response quality.
TD-TG5	Open-ended dialogue-response generation	Contextual appropriateness, helpfulness, consistency, safety, and conversational quality.
Automated research		
TD-AR1	Research-idea generation	Novelty relative to evidence, significance, feasibility, and testability.
TD-AR2	ML/NLP ablation-plan generation	Hypothesis identification, controlled contrasts, confound management, and feasibility.

Table A1 continued

ID	Domain	Core quality considerations
TD-AR3	Table-to-insight report generation	Numerical faithfulness, appropriate inference, uncertainty, and relevance.
TD-AR4	Related-work generation	Coverage, citation correctness, faithful comparison, and synthesis.
TD-AR5	Peer-review critique generation	Technical accuracy, evidence, significance assessment, fairness, and actionable criticism.

What existing resources actually supply. Table A2 distinguishes inputs and reference material from judgments on generated outputs. A published dataset name is not evidence that the required preferences, grouping identifiers, or permissions are available. In particular, a caption reference is not a preference between captions, and dimensional scores do not define an overall ordering without an explicit human standard. The source audit records URLs and versions separately from experimental completion.

Table A2: Source candidates and missing work, checked against public source descriptions. These are readiness records, not final dataset selections. A source used to train a released metric cannot simultaneously provide an unexamined independent test of that metric.

Domain	Candidate resource	Verified role	Work before use
SV4	Versioned specification/output pool	Local task definition; public source not yet frozen	Recover specifications, assets, behavior checks, and collect matched preferences
SV2	Design2Code (Si et al., 2025b)	Screenshots, HTML, generated outputs, released human comparisons	Join exact outputs and votes; audit sites and asset permissions; not an SV4 substitute
SV5/6	PPTAgent; PresentBench (Zheng et al., 2025; Chen et al., 2026b)	Presentation inputs, artifacts, or evaluation materials	Check standard compatibility, original groups, and availability of human labels
IG1	DOCCI (Onoe et al., 2025)	Images and human-written detailed descriptions	Generate or recover caption groups; collect overall and dimensional preferences; group related images
IG2	Pick-a-Pic; ImageReward (Kirstain et al., 2023; Xu et al., 2023)	Text-to-image preferences; ImageReward ranking resources	Check asset access, prompts, provenance, and metric-training overlap; not caption preferences
TG1	SummEval (Fabbri et al., 2021)	Multiple summaries per article and four-dimensional human scores	Recover original articles; validate dimensional conversion; obtain overall preferences if absent
AR1	Research-ideation human study (Si et al., 2025a)	Ideas, expert ratings, and written reviews	Preserve topic/reviewer dependence; validate comparable groups and converted labels
Other	Resource selection pending	Scope only	No inclusion until the same readiness audit passes

SummEval’s many summaries derive from only 100 source articles; splitting summaries rather than articles would overstate the independent sample size. DOCCI supplies related-image clusters and entity metadata that help identify dependence; split these groups together where appropriate. Research-idea ratings require attention to shared topics and reviewers, and do not automatically yield a dense overall ranking. For text-to-image tasks, GenEval supplies compositional checks (Ghosh et al., 2023), not a replacement for human overall preference. A resource without usable labels can still provide materials for new annotation. A failure to obtain sufficient independent groups is a readiness limitation, not a reason to reuse the test set.

Table A3: Domain freeze and readiness template. Repeat the core rows for every additional admitted domain. All counts, approvals, and execution checks are pending; no out-of-distribution dataset is required.

Domain	Standard / source version	Train / Val / Test groups	Feedback / N	Adapter / permission audit
SV4	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
IG1	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
TG1	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
AR1	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Other	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>

Within-domain assessment. Train, terminal Val, and final Test contain different original-input/source groups sampled under the same fixed quality standard. This is an independent within-domain test, not a claim that arbitrary distribution shifts are solved. The test is sampled without consulting mining scores and is not restricted to disagreement cases. Its selection rules, generator mixture, and duplicate thresholds are frozen before adaptation. All derivatives of the same document, website/template, image cluster, or research topic remain together at the declared dependence level. An optional source or generator shift must name what changes and verify that the human standard remains comparable. Harder cases alone do not define a distinct domain.

B HUMAN FEEDBACK AND PREFERENCE MINING

Human feedback has four levels: H_0 gives overall preferences; H_1 gives dimensional preferences; H_2 identifies error type, severity, and location; H_3 optionally supplies a free-form rationale. An H_0 -only corpus remains usable, but it supplies neither human dimensional judgments nor human explanations. New labels are not assumed to be more reliable than reused labels. Both feedback routes provide the same interface to evolution while preserving the source of each label (Table A4). A record can be written as

$$h = (x, y_a, y_b, q, p, a, \nu), \quad (4)$$

where q is overall quality or a dimension, p is a preference, tie, or abstention, a records annotator agreement, and ν records provenance.

Table A4: Feedback routes and accounting. Existing-label replay measures revealed-label efficiency; new annotation additionally measures human effort. Neither route may reveal unselected or final-test labels to the miner.

Route	Permitted evidence	Budget and missing information
Reused preferences	Original votes, rankings, or explicitly validated conversions on the exact outputs	Count revealed original-input groups and pair labels; retain unknown historical annotation time
New preferences	Blinded judgments on existing or freshly generated outputs	Count requested units, individual ratings, annotation minutes, abstentions, and adjudication
Both	Same feedback detail for a matched contrast; random anchor inside each budget	No model-generated rationale or author preference may fill a missing human field

Annotation and quality assurance. Freeze the rubric, visible evidence, tie policy, rater qualifications, compensation, consent, and institutional review determination before collection. The initial design uses two independent development judgments with adjudication as needed and three independent final judgments; final counts must be specified per domain rather than assumed from this template. Blind method and generator identity, model scores, and earlier winners, and randomize presentation order. Keep individual votes, dimensional ratings, ties, no-consensus outcomes, insufficient responses, and abstentions. Report valid-rater-pair agreement with its denominator and abstention over all requested ratings. A changed rubric or visible evidence is a changed protocol, not automatically proof that the old labels were wrong. Table A5 is filled separately for each route and domain.

Table A5: **Human-feedback audit.** Simulated ideal scenario; no human annotation was conducted. Agreement and abstention are assigned percentages, not rater-level estimates. Groups, raters, and person-minutes illustrate the reporting fields; route and individual-vote records remain unfilled.

Domain / route	Groups	Raters	Agreement	Abstention	Minutes
SV4	200	3	86.0	3.0	660
IG1	200	3	88.0	2.0	600
TG1	200	3	87.0	2.5	630
AR1	200	3	81.0	6.0	900

Label-blind selection. Deduplicate and partition original-input groups before mining. Freeze signal definitions and source mappings, select the random anchor without labels, and use only authorized anchor feedback for any human reliability qualification. Freeze the targeted sampling policy before revealing the remaining selected labels. For existing datasets, replay exactly this label-access order. The random sampling arm includes its anchor within its uniform budget, not as extra data. Both routes retain the full unselected pool for accounting; the miner never sees final-test labels. Human qualification is not inferred from agreement with a model-annotated pilot.

Implemented conflict ranking. Preference mining computes structural disagreement without reading the pair’s human preference. Outputs are grouped by their original task input and deduplicated before comparison. For a pair $r = (x, y_a, y_b)$, signal j supplies a preference sign $u_j(r) \in \{-1, 0, +1, \perp\}$ from its own score difference and declared tie margin. Zero denotes a tie and $\perp$ missing or abstaining output; raw scores from different signals are never subtracted. Signals require executable sources with compatible inputs; provenance, repeated-run variation, and missingness are recorded separately. We freeze a many-to-many mapping M_{jq} from signal j to dimension q using its definition and measurement procedure before reading preference labels. A mixed score may connect to several dimensions, but remains one shared signal. Signals sharing a model, training labels, or underlying metric are assigned source groups so that duplicated prompts do not become independent votes. Metrics reserved for final independent assessment remain outside mining, evaluator tools, critique, and reward.

The ranking distinguishes three cases. *Within-dimension disagreement* occurs when signals from distinct source groups favor opposite outputs on the same criterion. *Overall-dimension disagreement* occurs when an overall signal opposes a dimensional signal. *Cross-dimensional trade-offs* occur when dimension-level preferences favor different outputs; these are not contradictory measurements of one criterion. Writing their indicators as $I_{\text{within},q}$, $I_{\text{overall},q}$, and I_{tradeoff} , the implemented priority is

$$P(r) = 2 \sum_q I_{\text{within},q}(r) + \sum_q I_{\text{overall},q}(r) + I_{\text{tradeoff}}(r). \quad (5)$$

Pairs are ranked by decreasing $P(r)$, with stable pair identifiers breaking ties. The coefficients define a structural ranking, not estimated signal reliability or a probability of evaluation error. In particular, $P(r) = 0$ can reflect missing signals or ties rather than agreement, and agreement itself need not be correct. Overall-dimension comparisons retain whether their signals share a source group; they are not automatically independent evidence. Each source group contributes at most one preference sign to a dimension’s aggregate preference, with opposing within-group signs yielding abstention. Missing outputs and small-margin ties remain distinct from opposing preferences.

For repeated model signals, the pairwise signal margin is the median of same-repeat score differences between the two artifacts; variation and sign changes are retained. This differs from Equation 6, which first takes each artifact’s median final score and then compares them. A missing required signal repeat makes that evaluation signal unavailable for mining. Deterministic metrics use declared ranges and orientations, not ranges fitted to the output pool or human labels. The random anchor is selected by original-input hashes before scoring, preserving output groups. Label-blind execution does not establish label-blind upstream cohort selection; historical data must also document how the output pool was chosen.

Human qualification and implementation boundary. The implemented miner supplies the structural queue and its source-dependence records. The full preference-mining protocol addition-

ally requires human-annotated anchors to qualify signals before targeted annotation; cross-domain human-reliability calibration and the equal-budget advantage remain unverified. Model-annotated pilot anchors measure agreement with that model, not human reliability. The annotation fraction, qualification thresholds, and targeted policy must be frozen for the human study, rather than inherited silently from a pilot. The feedback table Z is a logical join over feedback, artifact, signal, and execution records; absent human dimensions or rationales remain absent.

Source readiness. A signal is marked executable only after its input contract, output orientation, dependencies, and failure behavior are verified. A publication, URL, or repeated prompt does not establish executable or independent evidence. Report missing dimensions and correlated source groups rather than creating artificial votes. External metrics reserved for assessment must not enter mining or the evolved evaluator.

C ACCESS BOUNDARIES, ENDPOINTS, AND UNCERTAINTY

Table A6 distinguishes development feedback from independent assessment. The deployed scorer always receives one output and permitted task evidence, without opponent outputs, human labels, generator identity, split names, or mining strata. Only the authorized optimization process sees proposal-Train labels and traces. Automated tools can inspect the artifact; they cannot retrieve held-out answers or invoke reserved assessment metrics.

Table A6: Information access shared by the controlled comparisons. A sealed development set is still adaptively used training evidence, not an independent test.

Partition	Released information	Permitted decisions
Proposal Train	Authorized artifacts, human feedback, detailed traces	Proposals, diagnoses, text edits, program edits, or fitting fusion
Sealed Train	Fixed aggregate agreement, coverage, and regression checks	Native candidate selection and subsequent search; no per-example labels
Terminal Val	Measurements for the Train-fixed shortlist	Assessment only; no reranking, new edits, or restart selection
Final Test	Locked until methods, checkpoints, and analysis are frozen	Reporting only; post-search checkpoint curves cannot guide search
Metric reserve	Endpoints excluded from optimization and runtime tools	Independent assessment of the relevant quality attribute

Published optimizers may retain their native development population or selection rules within this common access boundary. Their validation interface maps to development data, not to the locked final test. If an adaptation needs per-example feedback that the shared interface does not expose, it must use proposal Train or be reported as an unmatched condition. The terminal evaluator is selected by the declared Train-only rule before terminal Val is opened. Interrupted runs retain their failures and incurred costs; they do not receive a fresh selection opportunity.

Agreement estimator. Use the repeated-score protocol of the deployed evaluator, requesting $R \geq 3$ executions per artifact and freezing R , precision, retries, and cache identity before measurement. Let P_g be the strict human preferences within original-input group g . For complete executions,

$$\begin{aligned}
 \bar{s}_\theta(x, y) &= \text{median}_{r=1, \dots, R} s_\theta^{(r)}(x, y), \\
 v_{gj} &= \mathbb{I}[\bar{s}_\theta(x_g, y_{gj}^+) > \bar{s}_\theta(x_g, y_{gj}^-)], \\
 \hat{J}(\theta) &= \frac{1}{|\mathcal{G}|} \sum_{g \in \mathcal{G}} \frac{1}{|P_g|} \sum_{j \in P_g} v_{gj}.
 \end{aligned} \tag{6}$$

Input groups receive equal weight; predicted ties count as incorrect. Human ties do not create strict preference labels. Groups without a strict preference retain a separate count and do not enter this endpoint. The native development proxy can use one fixed pair per input; final evaluation need not use that reduced proxy. A reused artifact keeps its frozen repeated scores across all comparisons.

Missing execution and coverage. Fix weights $w_{gj} = 1/(|\mathcal{G}||P_g|)$ before execution. Let c_{gj} indicate complete scoring of both artifacts, $M_{\text{cov}} = \sum w_{gj}c_{gj}$, and $K_{\text{corr}} = \sum w_{gj}c_{gj}v_{gj}$. Report conditional agreement $K_{\text{corr}}/M_{\text{cov}}$, weighted coverage, raw counts, and

$$[K_{\text{corr}}, K_{\text{corr}} + 1 - M_{\text{cov}}], \quad (7)$$

the possible full-cohort agreement range under arbitrary missing outcomes. This is not a confidence interval. Main agreement contrasts use a common complete cohort across the compared methods, with the requested-cohort bounds alongside them; method-specific complete cohorts are supplementary. Missing required repeats or tool results remain missing, rather than being replaced by successful retries selected after seeing their score. Valid evidence of a low-quality artifact is an outcome, not an execution failure.

Selection and top-region judgments. Freeze a common output pool size for the main selection cohort; the initial design proposes $N = 4$, subject to data readiness. Other N values are separate sensitivity analyses. Complete human rankings, including declared ties, identify the set of top-ranked outputs. Best-of- N succeeds when the evaluator selects a member of that set. Resolve evaluator score ties by a fixed output-identifier rule independent of human rank. With human ties, define regret as the number of outputs strictly preferred to the selected output divided by $N - 1$; for strict rankings this is $(\text{rank}(\hat{y}) - 1)/(N - 1)$ for the selected output $\hat{y}$. For a human-ranked group without a valid evaluator selection, count selection as unsuccessful and regret as one, and additionally report complete-case sensitivity. Top-region agreement uses strict pairs touching the human top-two region, including any ties at its boundary. No-consensus or insufficient human rankings are reported separately and never supplied by the evaluator. Pair-only data cannot establish any of these multi-output endpoints.

Aggregation and uncertainty. Freeze the admitted domain set per endpoint before outcomes are opened and use equal-domain means. Report the domain count, input/source groups, pairs, outputs, ranking pool size, and completion for every number. Separate reused-label and new-label results before combining compatible endpoints. Do not pool incomparable quality metrics or replace an unavailable domain after observing its performance.

The main performance estimate averages independently completed searches, not the best search selected retrospectively. Static methods have no search-seed variance. The initial target is five independent searches per adaptive condition, with the final count and sample sizes frozen after a development-only feasibility and variance pilot. Report all run estimates when the available count is too small for stable intervals. For fixed evaluators, compute paired 95% bootstrap intervals by resampling original-input/source groups jointly across methods, retaining all their outputs. For optimizer comparisons, additionally resample independent search runs, paired only when the experimental blocks were deliberately paired; otherwise resample method runs independently. The same input resample is used across methods and runs. Scoring repeats do not increase the number of independent tasks or searches. If shared reviewers introduce additional dependence, retain reviewer identifiers and report reviewer-cluster or crossed-cluster sensitivity rather than treating votes as independent.

Predeclare one principal program comparison (IterEval versus GEPA), the Seed contrast, and the direction-versus-history contrast. Report paired effect sizes and intervals for all other comparisons, with a prespecified multiplicity correction for any family of significance claims. Select sample sizes from a target minimum effect and development-only variance, not from a desired test outcome. A nonsignificant difference does not establish equivalence. Bootstrap intervals on one final evaluator do not establish search stability.

Independent checkpoints. Record each search’s best accepted program at fixed fractions of the cost cap, proposed as 25, 50, 75, and 100%. Use the last completed selection before each boundary. Assess these frozen checkpoints on Test only after all search and configuration choices are complete. A run ending early carries its last accepted program forward, with its stopping reason retained. Do not interpolate unexecuted programs or use the test curves to pick another final evaluator.

D BASELINES, ADAPTATION FIDELITY, AND RESOURCE MATCHING

Table A7 defines the main rows by what is optimized and why they are needed. An official repository establishes availability, not a completed reproduction on these tasks. Before running, pin its commit, inspect the adapter, verify label access and model routing, and record any departure from the published implementation. The literature and source audit supporting these choices is maintained with the experiment manifest.

Table A7: Main baseline contract. All adaptive rows use the same permitted human feedback. Native search semantics are preserved; differing editable spaces are explicit.

Method	Optimized object	Required fidelity and interpretation
Fixed model judge	None; fixed prompt and base model	Pointwise score on the artifact, no extra tool inspection; a static reference, not a matched observation ablation
Seed evaluator	None; initial full evaluation program	Same seed, tools, routes, and score interface used to initialize program evolution
Learned signal fusion	Weights over fixed signal features	Fit on proposal-Train preferences; hold features, model, and observation path fixed
SkillOpt (Yang et al., 2026)	Skill text	Preserve bounded edits, validation gate, rejected-edit buffer, and specified meta updates; pin paper-style variant
GEPA (Agrawal et al., 2026)	Full program through a declared adapter	Use official engine and program-capable interface; preserve reflection, population selection, and configured merge policy
Meta-Harness (Lee et al., 2026)	Harness code	Preserve filesystem access to prior code, scores, and traces; do not restrict parents to the current best
IterEval	Full evaluation program	Native active direction, diagnostics, acceptance, and conservative parent rule

Learned signal fusion. A simple reproducible control fits a regularized pairwise logistic model to differences in frozen per-output signal features on proposal Train. Use its pointwise linear score for final ranking. Freeze feature orientation, normalization, missing-value handling, regularization search, and dimensional-label use before testing. Fit transformations only on training data. A monotonic remapping of one score cannot improve its ranking except through altered ties; it is not a meaningful substitute for this multi-signal control.

SkillOpt. Use the official text/skill optimization path, not a custom program-evolution extension bearing its name. The candidate skill is inserted into a fixed seed interface, with tools and routes unchanged. Retain rejected edits and native longer-timescale updates; do not describe the baseline as memoryless or assume every failed edit causes a semantic restart. Map its internal selection feedback to development data and charge reflection and selection calls. Compare it additionally with IterEval restricted to the identical text fields, evidence inputs, and edit envelope. This same-space contrast is necessary to attribute a difference to optimization rather than broader program access.

GEPA and Meta-Harness. Current GEPA interfaces support program optimization; restricting it to prompts would not test the closest alternative. Use a declared whole-program or multicomponent representation with the official optimizer, returning scores and authorized trajectories through its evaluator adapter. Preserve the configured population, candidate-selection, and merge mechanisms, and distinguish this program adaptation from the original paper’s particular experiments. Meta-Harness must retain its full-history code and trace access. Its released examples assume a particular coding-agent wrapper; replace only the transport needed for the matched optimizer model, preserving logging and native search decisions. If model access cannot be matched, place that run in an explicitly unmatched supplementary comparison rather than the main controlled result.

Additional comparators. AFlow is the first optional structured workflow-search baseline because it provides an explicit workflow search implementation (Zhang et al., 2025). Its multimodal task adapter must be validated rather than assumed from text/code examples. ADAS and DGM are relevant architecture/code-evolution alternatives (Hu et al., 2025; Zhang et al., 2026), but need not all enter the main table if the closest full-program comparisons are already covered. HarnessBank and

SkillHEX remain related methods unless an official executable release and compatible evaluation interface are verified (Luo et al., 2026; Feng et al., 2026). A paper or abstract is not sufficient to mark a baseline implemented.

Evaluator fine-tuning is an optional separate study on an open-weight backbone, not a required row across domains with incompatible model access. It follows the trained-judge direction exemplified by Prometheus and CritiqueLLM (Kim et al., 2024a; Ke et al., 2024). Match the initial checkpoint, feedback, observations, and training-selection access; report training compute separately. No additional teacher labels are free. This trains the evaluator, unlike C3, which trains the generator. A query-specific rubric model trained for research reports (Lv et al., 2026) is not automatically an ideation evaluator.

Resource controls. The primary evolution cap is total model-and-tool cost at a frozen price schedule. Count proposal and reflection tokens, diagnostics, rejected and invalid attempts, repeated candidate measurements, retries, and the reserved terminal assessment. Track actual tokens, calls, accelerator time where applicable, wall time, and tool-specific charges separately. Different attempt counts under equal cost are outcomes. Mining and annotation are separate cost categories; the end-to-end comparison includes both. Deploy every final evaluator under the same permitted input, context, tool, time, and token caps. Report actual deployment cost instead of assuming that an equal cap produces equal use. Fixed final evaluation costs do not buy additional search. Table A8 records these distinctions.

Table A8: **Adaptation and deployment resources.** Simulated ideal scenario; not measured costs. Search cost is in relative units (optimizer cap = 100), tokens in millions, deployment cost relative to Seed, and latency in seconds. Actual reporting includes failed and diagnostic calls, separately by domain and run.

Method	Search cost	Search tokens	Deploy cost	Latency
Fixed model judge	N/A	N/A	0.25	2.0
Seed evaluator	N/A	N/A	1.00	8.0
Learned signal fusion	5	0.04	1.03	8.2
SkillOpt	100	0.9	1.05	8.5
GEPA	100	1.1	1.15	9.4
Meta-Harness	100	1.2	1.20	10.1
IterEval	100	1.1	1.18	9.8

E DEPTH-FIRST EVALUATOR EVOLUTION AND ATTRIBUTION CONTRACT

Figure A1 separates the editable observation and judgment mechanisms. Algorithm 1 specifies their evolution and selection. Self-evolution means automated changes to this evaluation program under external human feedback, not self-definition of the human standard.

Investigation and candidate construction. The evolution agent reads original proposal-Train artifacts, installed component instructions, execution traces, and relevant history through a tool-driven editing session. It can request additional authorized proposal-Train measurements or fixed-input component probes. The submission contains actual edits, a short intent, and references to inspected training cases. Predicted orderings, contrast cases, and intermediate checks are optional declarations, not controller-inferred scientific predictions. The controller records the exact parent, installed program, and measured outcomes. Local diagnostic success is not an admission requirement for the common development measurement. The state (B, W, d, z) in Equation 1 summarizes these persisted records. The direction identifier links its plan, root errors, previous trials, and unresolved questions. Root errors are evaluation errors present in the program from which the direction starts. Starting a new direction does not delete earlier direction histories. The native patch interface enforces the direction identifier, but allows the intent to be revised; it does not enforce an invariant target set or semantic correction.

An elementary edit changes one component; a connected candidate can assemble a tool, its route, and its consumer against a fixed parent. Only the complete candidate is executed and measured,

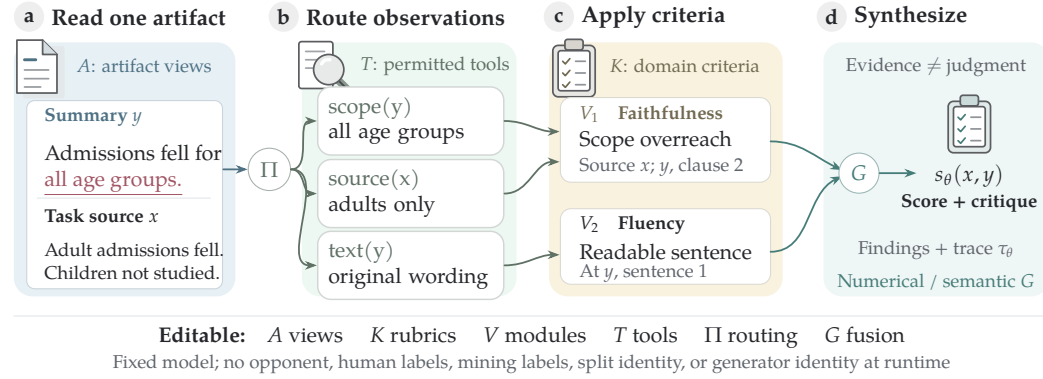


Figure A1: The six-component evaluation program. (a) Read one output and permitted task evidence. (b) Route tool observations. (c) Apply dimensional criteria. (d) Fuse sourced findings into a score and critique. The lower rail identifies the editable components. Human feedback guides evaluator evolution, never the deployed scorer. Artifact evidence and judgment remain separate; the summary and findings are a constructed example.

not its partially assembled versions. Changed tools must be connected to valid routes, although valid routing alone does not establish correct use of their outputs. Interface validity, fixed-model constraints, and execution limits remain mandatory; edit-character counts are recorded as guidance rather than a hard acceptance or stopping rule. Unchanged programs and verified historical replays are distinguished from fresh trials, and failed calls remain in the cost record.

Diagnostic results. The diagnostic state retains execution records and repaired, unchanged, regressed, or unknown local outcomes, alongside the agent’s intended changes, interpretations, and unresolved questions. Recorded outputs are distinguished from explanations of why they occurred. These descriptions need not form a complete structured checklist for every candidate. Human preferences establish the target ordering; the agent must investigate why the program fails to reproduce it. Fixed-input probes compare supported specialist or synthesis variants while holding their upstream inputs fixed. Such probes bind their observations to the exact tested candidate and inputs; changing a tool, route, or topology requires the corresponding complete execution. An optional exact intermediate check records whether a declared field changed to its expected value on at least one complete repeat. All repeat-level matches, contradictions, and unknowns remain visible. A semantic expectation is not automatically verified, and a changed recorded field is not proof of a correct causal explanation or improved overall judgment.

Candidate acceptance and the next working program. Candidate acceptance requires complete, comparable execution, a strict development-agreement gain over the best evaluator, nondecreasing coverage, and the prespecified worst-stratum guard. These are development selection rules rather than statistical significance claims. Within a continuing direction, the parent rule determines the next working program. Using the candidate as the next parent first requires known, complete, comparable local and development outcomes, including comparisons against both the parent and best program. Subject to that requirement, accepted candidates and candidates with no observed damage can become the next parent. For this decision, damage includes a local ordering regression, deterioration on a declared protection case, or more regressed than corrected sealed-Train comparisons against either the parent or the best evaluator. An unaccepted candidate with damage is eligible only when fresh measurement supplies new progress and the declared protection cases do not deteriorate.

Qualifying progress is either a first-time repair of an error present at the direction’s root, still wrong in the parent, or a newly observed, predeclared exact intermediate-field change. Previously credited repairs and checks are retained across attempts, so breaking and repairing an already corrected case cannot renew this allowance. A missing protected case is unknown, not preserved. When the candidate fails these checks, the next parent returns to the pre-update best program, even if the candidate has just been accepted as the new best. The candidate and its diagnostics remain available. Keeping an intermediate candidate never changes the recorded best score.

Continuing and ending a direction. In the native editing session, the agent can request diagnostics, submit a candidate, repair an invalid submission, or end the active direction. Ending a direction uses the native `pause` action, which submits a no-change record with a nonempty reason. The session instruction asks for an unresolved question and why no different informative test is ready; the parser does not require a particular reason category or verify the explanation. The controller enforces the parent rule, validity, and resource limits without judging whether a direction is unhelpful. Revisions continue within the active direction until the agent ends it or an execution or resource limit stops the search. The history is retained in either case. An ending explanation may concern implementation problems or results suggesting that an implemented correction does not address the target errors. That explanation is the agent’s assessment, not an automatically verified conclusion. The current implementation therefore does not prevent an agent from ending a direction early or guarantee a minimum exploration depth. One unsuccessful implementation, a nonsignificant difference, or budget exhaustion alone does not establish that the direction is unhelpful. In the default implementation, edit size and cumulative direction edits are guidance, not a hard direction budget. Edit-character counts, replacement fractions, and component counts are advisory. Configured proposal opportunities, session action bounds, and execution limits constrain the run; they should not be described as an implemented fixed depth per direction. An integrity interruption does not trigger the normal terminal-validation procedure.

Terminal assessment and attribution. The terminal shortlist includes eligible, previously accepted, complete programs with positive development gain over the seed and is ordered using Train evidence alone. Its membership and order are frozen before Val access. Terminal Val does not rerank the shortlist or guide another proposal. With no eligible improvement, the seed is retained without an additional Val query. Fresh system confirmation and independent held-out assessment remain separate from adaptive development.

Connected edits share one parent and one measured outcome, so a bundle’s gain is not independent evidence for each edit. Component attribution requires paired interventions with the remaining interfaces fixed, such as tool on/off or evolved versus seed synthesis. If removing a dependency makes the program invalid, the comparison must report that limitation rather than substitute an unrelated whole-program control. Complete prompt templates, interaction traces, candidate changes, and execution settings belong in the reproducibility record rather than the main algorithm.

F SEMANTIC SYNTHESIS AND ANCHORED SCORING

The default fusion procedure forms an overall judgment from specialist findings; a declared numerical rule is an explicit alternative. Each specialist produces findings with identifiers, supporting evidence, dimensions, and uncertainty, as well as its own rating and reason. The full specialist trace remains available for diagnosis, but the synthesizer’s default input omits the structured specialist score, rating, and score scale. It retains the findings, reasons, and scope of each specialist and applies its own overall scale. This input separation does not remove value judgments expressed in the reasons.

The synthesizer groups findings by their consequences for quality, including strengths, weaknesses, mixed effects, and uncertainty. Every finding reference must occur in exactly one group. This accounting rule makes omitted or multiply referenced findings detectable; it does not guarantee correct semantic grouping. Duplicate descriptions should not become independent penalties, and favorable style should not cancel a factual error when the domain standard prioritizes faithfulness.

Let $a_k < a_{k+1}$ be adjacent overall anchors and $\lambda \in [0, 1]$ locate quality between them. The resulting scalar is

$$s = (1 - \lambda)a_k + \lambda a_{k+1}. \quad (8)$$

The anchors have domain-specific verbal meanings; specialist scores with different meanings are not averaged. Reference completeness, semantic correctness, and calibration require separate checks. A synthetic arithmetic example uses $a_k = 50$, $a_{k+1} = 75$, and $\lambda = 0.4$, giving $s = 60$. These numbers illustrate interpolation, not a universal scale or an evaluator result.

G DETAILED C1 RESULTS AND SELECTION DIAGNOSTICS

Table A9 supplies the per-domain records behind Table 1 and Figure 3. Fill one block per admitted domain, feedback route, and ranking-pool size. Report all seven main methods, with independent-run means, paired contrasts, run counts, and both requested and completed group counts. Do not replace the same-domain comparison with different domains for different methods.

Table A9: **Per-domain C1 comparison.** Simulated ideal scenario; all group and search counts are assigned. Pair, top-region agreement (Top), and Best-of- N accuracy are percentages; Reg is normalized rank regret. Dimensional outcomes (Dim.) remain unassigned. No per-run observations or confidence intervals are implied.

Domain / method	Pair	Top	BoN	Reg	Dim.
<i>SV4; N = 4, 160 groups, 5 searches</i>					
Fixed model judge	62.5	58.5	41.5	0.333	not assigned
Seed evaluator	68.0	63.5	48.0	0.288	not assigned
Learned signal fusion	70.0	65.6	50.5	0.267	not assigned
SkillOpt	73.0	69.0	54.5	0.238	not assigned
GEPA	75.5	71.7	57.5	0.216	not assigned
Meta-Harness	76.0	72.2	59.0	0.210	not assigned
IterEval	77.5	73.7	60.5	0.200	not assigned
<i>IG1; N = 4, 160 groups, 5 searches</i>					
Fixed model judge	65.0	59.5	42.5	0.328	not assigned
Seed evaluator	70.0	64.5	49.0	0.283	not assigned
Learned signal fusion	71.5	66.6	51.5	0.262	not assigned
SkillOpt	74.0	70.0	55.5	0.233	not assigned
GEPA	77.0	72.7	58.5	0.211	not assigned
Meta-Harness	77.5	73.2	60.0	0.205	not assigned
IterEval	79.0	74.7	61.5	0.195	not assigned
<i>TG1; N = 4, 160 groups, 5 searches</i>					
Fixed model judge	66.0	60.5	43.5	0.323	not assigned
Seed evaluator	71.0	65.5	50.0	0.278	not assigned
Learned signal fusion	73.0	67.6	52.5	0.257	not assigned
SkillOpt	75.0	71.0	56.5	0.228	not assigned
GEPA	78.0	73.7	59.5	0.206	not assigned
Meta-Harness	78.5	74.2	61.0	0.200	not assigned
IterEval	80.0	75.7	62.5	0.190	not assigned
<i>ARI; N = 4, 160 groups, 5 searches</i>					
Fixed model judge	61.5	61.5	44.5	0.318	not assigned
Seed evaluator	67.0	66.5	51.0	0.273	not assigned
Learned signal fusion	69.5	68.6	53.5	0.252	not assigned
SkillOpt	71.0	72.0	57.5	0.223	not assigned
GEPA	74.5	74.7	60.5	0.201	not assigned
Meta-Harness	75.0	75.2	62.0	0.195	not assigned
IterEval	75.5	76.7	63.5	0.184	not assigned

Selection diagnostics. Report the full selected-rank distribution, including scoring failures, and prespecified strata for omission, factual error, and competing quality criteria where meaningful. Define strata before seeing comparative outcomes and retain small or unavailable strata as such. Do not infer selection gains from global pairwise agreement. The main selection cohort uses one common N ; a sensitivity sweep changes N only on groups with enough independently ranked outputs. When comparing dimensional scores with overall preferences, preserve their separate constructs.

Cost and cross-task reporting. For empirical reporting, Figure 3b requires actual deployment cost per requested output from Table A8, not search cost. The current draft instead uses assigned relative costs. Panel (c) uses the same C1 values as Table 1, but answers where gains occur rather than who has the highest absolute mean. Real-data replacement must compute paired differences on

common inputs. This reuse is intentional; it must not be presented as another independent experiment. The external comparator is GEPA, fixed before inspecting its performance. Any missing task or comparator is labeled, not dropped to improve the plot.

H SEARCH, MINING, AND COMPONENT ABLATIONS

Table A10 prioritizes comparisons that identify the optimizer’s mechanism. The core ablations share feedback, seed, model, editable interface, measurement, and total cost cap. Secondary studies are explicit extensions rather than prerequisites for every domain.

Table A10: Ablation plan and supported inference. Core contrasts are required for the corresponding mechanism claim; secondary contrasts are scoped extensions. No outcome is assumed.

Factor	Conditions	What it identifies
Search (core)	Full; active direction with best-parent reset; history-only best-parent; no-history best-parent	Parent choice, then direction priority under a fixed parent policy, then history
Mining (core)	Random versus disagreement-based preference mining	Value of selected feedback at equal budget and fixed evolution
End-to-end (core)	Random/mined feedback $\times$ history-only/full evolution	Combined benefit and interaction; not two independent claimed gains
Editable scope (core)	SkillOpt versus text-only IterEval; full IterEval separately	Search within one edit space versus the effect of expanding that space
Evidence use (core)	Compatible tool-on/off and synthesis interventions	Whether measured evidence reaches judgment; effects need not be additive
Mining alternatives	Judge uncertainty; cross-model disagreement; repeated-run instability	Whether structural mining improves over simpler selection proxies
Feedback detail	Same groups with H_0 , $H_0 + H_1$, then available H_2/H_3	Marginal value at equal groups and measured human time
Architecture	Frozen observations; routing, rubric, or fusion interventions	Conditional contribution under fixed compatible interfaces
Data conversion	Direct versus converted judgments on the same subset	Adapter fidelity and downstream sensitivity
Optional extensions	AFlow; evaluator fine-tuning; source shift; direction reopening	Additional comparison, not an implemented or completed result

Search controls. Full IterEval uses its native active direction and conservative parent rule. The *active-direction*, *best-parent* control keeps direction priority and all diagnostics, but starts the next revision from the current best after a rejection. The *history-only*, *best-parent* control retains its full authorized history and proposes anew from the best program each round, without an active direction receiving priority. The *no-history*, *best-parent* control has the same current measurements but cannot inspect earlier failed attempts. Thus full versus history-only changes both priority and parent policy; it is not a pure priority ablation. Full versus active-direction, best-parent tests retaining intermediate implementations. Active-direction, best-parent versus history-only tests priority with the parent policy fixed. History-only versus no-history tests diagnostic memory. All are internal controls, not descriptions of SkillOpt, GEPA, or Meta-Harness.

Figure 4c partitions directions into four mutually exclusive outcomes. A direction with any accepted improvement is counted as a gain. Among directions without a gain, resource stops are counted separately; agent-ended directions are classified as early endings after at most one revision or endings without gain after multiple revisions. These outcomes describe the record, not whether the agent’s explanation was correct.

Mining budgets and policies. The tentative group-budget grid is 20, 50, 100, and 200, truncated or replaced before execution if the domain lacks enough independent groups. These are design values, not measured sample counts. Keep the seed, feedback detail, evolution cost, and final test

Table A11: **Matched-cost search comparison.** Simulated ideal scenario; not observed searches. Agreement is in percent; difference is from full IterEval in percentage points. Revisions per direction and search cost (cap = 100) are assigned aggregates, without run-level uncertainty.

Condition	Agreement	Difference	Revisions/direction	Cost
No-history, best-parent	72.6	-5.4	1.0	100
History-only, best-parent	75.0	-3.0	1.0	100
Active direction, best-parent	77.5	-0.5	3.2	100
Full IterEval	78.0	0.0	3.8	100
Text-only IterEval	74.3	-3.7	3.0	100

fixed across policies. Restart evolution from the seed at each budget; a warm-start sweep answers a different question. Use nested selected sets only if that policy is frozen in advance, retaining their dependence in uncertainty estimates. A revealed original-input group is the basic label-access unit, but also report the number of pair labels and ratings so large groups do not provide hidden extra feedback. New-annotation curves additionally use measured person-minutes, including anchors, extra raters, adjudication, and abstention. Existing-label replay cannot claim those minutes were saved unless comparable time records exist. If H_1 or H_2 differs across routes, do not attribute its effect to the sampling policy.

The secondary selection policies use prespecified judge margins, disagreement across source groups, or repeated-run sign instability. They draw from the same deduplicated output pool without unselected human labels. Freeze source grouping, uncertainty direction, tie thresholds, random seeds, and anchor size before comparing policies. An unknown signal is not a confident agreement. Charge pool scoring and reliability checks separately from evolution.

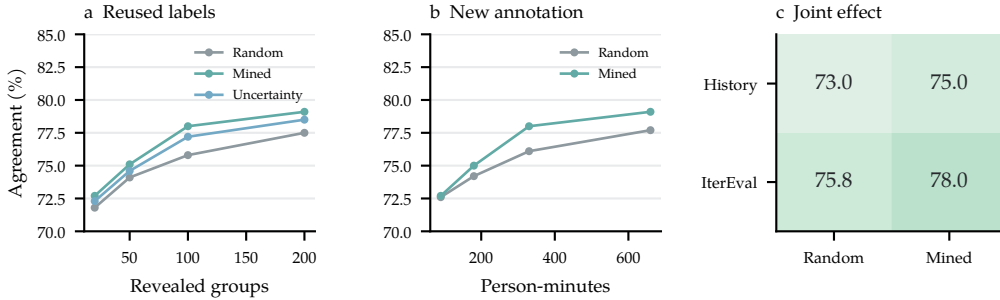


Figure A2: **Preference-mining efficiency.** Simulated ideal scenario; not annotation or search measurements. (a) Held-out agreement versus revealed groups in existing-label replay. (b) Agreement at matched person-minute budgets for new annotation. (c) Random/mined feedback crossed with history-only/full evolution at 100 groups. All curves and times are assigned without statistical inference; empirical curves require independent selection/search runs and uncertainty.

Compatible component interventions. Hold input evidence fixed when comparing seed and evolved interpretation or synthesis. When removing a tool or changing a route changes available evidence, rerun the complete program under the common budget. A dependency-breaking ablation is invalid, not a low score. Use compatible seed replacements and record when no meaningful intervention exists. Include tool on/off, seed/evolved tool, fixed/evolved routing, fixed/evolved rubric and modules, and numerical/semantic fusion where the seed supports them. A connected candidate's overall gain cannot be assigned independently to all edited components. Table A13 reports paired effects rather than an additive decomposition.

Table A12: **Mining and end-to-end comparison.** Simulated ideal scenario; not annotation or search results. The first four rows form the two-by-two design at 100 revealed groups. Agreement is in percent; minutes describe a separate new-annotation scenario. Total model-and-tool cost includes mining and search in relative units, not human labor.

Feedback	Search	Groups	Minutes	Agreement	Total cost
Random	History-only	100	300	73.0	102
Mined	History-only	100	330	75.0	106
Random	IterEval	100	300	75.8	102
Mined	IterEval	100	330	78.0	106
Uncertainty	IterEval	100	318	77.2	104
Cross-model	IterEval	100	330	77.6	106
Run instability	IterEval	100	345	76.9	108

Table A13: **Component and feedback diagnostics.** Simulated ideal scenario; no interventions were executed. Agreement differences are percentage points. Cost differences use Seed-normalized deployment cost for program interventions and person-minutes for feedback-detail comparisons. These assigned effects are not additive or accompanied by statistical inference.

Contrast	Held fixed	Groups	Agreement difference	Cost difference
Tool on/off	Compatible consumers	160	+3.2	+0.12
Evolved/seed tool	Remaining program	160	+1.8	+0.05
Evolved/fixed routes	Tool inventory	160	+1.3	-0.03
Evolved/seed judgment	Artifact evidence	160	+2.6	0.00
Semantic/numerical fusion	Specialist findings	160	+2.1	+0.02
$H_0 + H_1 / H_0$	Same labeled groups	100	+2.4	+30 min
Additional H_2 / H_3	Same groups	100	+1.0	+65 min
Direct/converted labels	Same protocol	160	+0.4	N/A

I TRAJECTORY AND FAILURE ANALYSIS

Figure A3 connects the recorded search state to actual behavior. For every direction, retain its starting error cases, intended correction, candidates, parent programs, observations, agent explanations, selection outcomes, cost, and end reason. The implementation’s direction identifier can persist while its description changes; human review must therefore check semantic continuity rather than infer it from an identifier. Record early agent endings, semantic drift, successful corrections, failed directions, and resource stops.

a Continued correction	b Unsuccessful direction
Target error and intended change.	Target error and attempted changes.
First implementation: what did execution show?	Which observation remained missing or unhelpful?
Later revision: what changed and was accepted?	Why did the direction end, or run out of resources?
PLACEHOLDER — retain actual parent links, costs, and unknown outcomes; do not invent a success case.	

Figure A3: **Observed trajectory and failure audit (unmeasured).** (a) A direction with an initially unaccepted candidate and later improvement, if one exists. (b) A direction that ends without improvement or exhausts resources. Show parent links, actual edits, changed observations, local errors, development agreement, acceptance, and cumulative cost. Separate recorded outcomes from agent interpretations. Cases follow a fixed sampling rule; no favorable trajectory is assumed to exist.

Case selection and interpretation. Choose trajectories using a declared rule over outcome categories and stable identifiers, not by visual appeal. Review a random sample of all directions and separately sample any later-success, no-gain, and exhausted categories. Where feasible, reviewers assess intended corrections and observable behavior without knowing the comparative final scores. A

representative case must identify what was missing in the earlier implementation and what changed later. Improvement after several revisions is temporal evidence; the controlled ablations are needed to attribute a benefit to continuation. Ending a direction is the agent’s decision, not verified evidence that the correction was impossible.

Failure taxonomy and reporting. Classify observed missing evidence, unused tool outputs, interpretation errors, incompatible edits, and uncertain diagnoses. Allow multiple causes and unknown status. Report the share of directions with no accepted candidate, early endings, repeated equivalent edits, and tests that added no new observation. Distinguish technical failure, lack of measured gain, and unresolved questions at the budget limit. These analyses explain costs and limitations without turning plausible explanations into causal facts.

J DOWNSTREAM REFINEMENT AND INDEPENDENT QUALITY CHECKS

C2 tests the usefulness of critique, not whether the evaluator awards higher scores to its own revisions. On the same unseen inputs, generate a new y_0 and retain it unchanged as a reference arm. Compare no-critic revision, Seed-critic revision, and IterEval-critic revision. Use the same generator, decoding, revision instruction, critique format, evidence access, token cap, and number of revision steps. Freeze these settings on input-disjoint development material before final generation. A supplementary critique from a learned baseline can be added only with the same interface and accounting.

New blind annotation assesses the exact newly generated outputs. Existing benchmark preferences do not label these revisions. The main contrast is IterEval-refined versus Seed-refined; comparisons with no-critic revision and y_0 test extra-compute and nondegradation explanations. Report wins, losses, ties, abstentions, invalid outputs, and the fraction with a major regression relative to y_0 . The tie-aware preference estimate is $(W + T/2)/(W + L + T)$, with a separate requested-input denominator and bounds for unjudged outcomes. Major regressions are defined by the domain rubric before annotation, not by whichever errors remain after revision.

Table A14: **Downstream controls and per-arm outcomes.** Simulated ideal scenario; no outputs, votes, or training runs were collected. Preference, major regression, and invalidity are percentages. Regression describes the first arm relative to y_0 (C2) or Base (C3), not necessarily the row’s comparator. Counts are aggregate assignments with no paired intervals.

Contrast	Win / tie / loss	Pref.	Major regression	Invalid	Judged / requested
C2: Seed / y_0	252/48/180	57.5	10.0	1.0	480/480
C2: IterEval / y_0	270/52/158	61.7	7.5	0.8	480/480
C2: IterEval / Seed	245/48/187	56.0	7.5	0.8	480/480
C2: IterEval / no critic	260/42/178	58.5	7.5	0.8	480/480
C3: Seed / Base	80/24/56	57.5	12.5	1.3	160/160
C3: IterEval / Base	86/24/50	61.3	10.0	0.9	160/160
C3: IterEval / Seed	80/20/60	56.3	10.0	0.9	160/160

Protected attributes and reserved metrics. Human preferences establish the overall endpoint. Table A15 adds domain-appropriate checks for particular attributes; none is assumed to capture overall quality. A metric used in mining, search, runtime evidence, critique, or reward is an optimization signal, not a reserved independent endpoint. If no suitable independent metric exists, record that absence and retain human dimensional judgments. Do not invent an ensemble score across incompatible constructs. When human and metric outcomes differ, inspect the same outputs and the metric’s scope before claiming either reward exploitation or human error.

K CONDITIONAL REWARD-GUIDED TRAINING STUDY

C3 remains design-only until separately authorized training and independent assessment are completed. Its purpose is to test whether changing the frozen evaluator reward improves a generator. It

Table A15: **Independent attribute checks (unmeasured)**. Freeze the exact implementation and reserve status before evaluation. Metric availability and scope are not inferred from a task-family name.

Domain	Target attribute	Candidate checks and limitations	Outcome
SV4	Requirements and usability	Held-back functional/specification checks plus human usability; passing code is not full visual quality	—
IG1	Grounding and useful detail	Source-supported claims, hallucination, and coverage; image-generation rewards are not caption metrics	—
TG1	Faithfulness and coverage	Source-support and key-content checks with human review; lexical overlap is supplementary	—
AR1	Novelty, feasibility, significance	Expert review and citation verification; no universal automatic novelty oracle	—
SV2	Reconstruction fidelity	Reserved render/structure checks; resemblance does not establish functional quality	—
IG2	Prompt correspondence	Applicable reserved compositional checks; protect visual quality with human judgments	—
TG4	Stated constraints	Applicable executable checks plus substantive human quality	—

does not establish that program evolution replaces evaluator fine-tuning or that a reward with higher offline agreement is safe to optimize.

Preflight. IG1 is the initial candidate; SV2 and a TG4 constraint-following control are optional extensions after feasibility checks. AR1 is not a priority for the first training study because automatic proxies do not replace expert evaluation of research quality. Before updating policies, score a common pool generated by the actual starting policy using both frozen evaluators. Measure agreement near the top of the pool, reward variance, ties, invalid responses, and sensitivity to length, formatting, omission, and unsupported detail. Freeze numerical tolerances and the stop/expand decision using development data. Failed preflight remains a reported result; it is not removed by substituting another task after testing.

Table A16: C3 execution manifest and outcomes. All hyperparameters and outcomes are pending. The study requires a frozen reward, matched policy-training resources, and new human endpoint judgments.

Item	Required specification or result
Arms	Untrained Base; Seed-GRPO; IterEval-GRPO; same initial checkpoint
Policy and data	Checkpoint, input source/version, source-group splits, decoding: <i>TBD</i>
LoRA / optimizer	Rank, target layers, optimizer, learning rate, KL rule, update count: <i>TBD</i>
Online rollouts	Same input sequence and group size; each evolving policy generates its own rollouts
Reward	Immutable program/tools, scale map, normalization, tie and invalid-output rules: <i>TBD</i>
Resources	Matched update resources and reward-call cap; measured tokens, tool cost, accelerator time: <i>TBD</i>
Preflight	Validity, variance, preference checks, exploitation probes, numerical gates: <i>TBD</i>
Checkpoint choice	Input-disjoint training validation and fixed selection rule; no final-test access
Final outcomes	New blinded preferences, dimensional quality, validity, regressions, reserved checks: —
Safety diagnostics	Length/format shifts, zero-variance groups, reward versus human-quality divergence: —

Updates and assessment. Base receives no policy updates; both trained arms start from that same checkpoint and use the matched LoRA/GRPO setup. Only the preflight pool is shared: once policies diverge, each samples its own online trajectories. Do not reuse one arm’s outputs as if they were on-

policy samples from the other. Freeze checkpoint-selection rules and evaluate the selected policies on the same previously unseen within-domain inputs. Collect new blinded judgments on Base, Seed-GRPO, and IterEval-GRPO outputs. Report the reward contrast as well as each trained arm against Base, with major regressions and external checks. An increase in optimized reward accompanied by unchanged or worse human quality does not support training usefulness. Additional training algorithms are optional; they answer a different question from changing the reward evaluator.

L OPTIONAL EXTENSIONS AND INTERPRETATION BOUNDARIES

Optional studies do not block the core domain comparison. Table A17 keeps their status explicit so that a missing extension cannot silently become a favorable result. Each needs a frozen question, resource contract, and independent endpoint.

Table A17: **Optional study register (not executed)**. Fill only after the study-specific prerequisites are met. N/A indicates an inapplicable comparison; a dash indicates an unmeasured one.

Study	Prerequisite	Interpretable contrast	Result
AFlow	Verified domain adapter and official code	Structured workflow search at matched resources	—
Evaluator	Common open checkpoint and observation interface	Weight adaptation versus restricted/full program evolution	—
Source/generator shift	Named shift with unchanged human standard	Transfer of a frozen domain evaluator; separate from primary test	—
Larger output pools	Human rankings for every included output	Selection sensitivity by N ; no mixed- N headline	—
Direction reopening	Implemented and frozen reopening policy	Reopening versus native nonreopening search	—
Additional C3 domain	Reward preflight and training authorization	Task-specific reward usefulness	—

Interpreting the completed experiments. If overall agreement improves without top-region or selection gains, limit the claim to agreement. If improvements occur only on the adaptively checked training set, the experiment has not shown held-out improvement. If history-only search matches full IterEval, report that direction continuation did not demonstrate added value at that budget. If learned fusion matches full evolution, the tested errors may not require changing the observation path. If mining wins per revealed label but not per person-minute, report label efficiency without claiming greater annotation efficiency. If C2 improves over Seed critique but not over no-critic revision, additional revision resources remain an alternative explanation. If C3 raises reward without better human judgments, do not infer better generation from evaluator agreement. Null, negative, and incomplete outcomes each have distinct reporting statuses.

M FIGURE AND TABLE COMPLETION INVENTORY

Table A18 is the completion checklist for the active manuscript. Figure 1 and Figure 2 remain conceptual and require no invented numerical results. Numerical displays use an ideal aggregate scenario to fix their layout; empirical records remain unfilled. Observed trajectories, dataset readiness, permissions, and execution manifests are not simulated. The companion fill-in plan specifies the record schema, frozen choices, and interpretation boundary for each item.

Table A18: Complete display inventory. Core empirical items are pending; optional entries are conditional. Appendix numbering is independent of main-text numbering.

Display	Question / role	Required inputs
Figs. 1, 2	Intuition and method	Existing artwork and constructed-example disclosures; no result claims

1674	Table A18 continued		
1675	Display	Question / role	Required inputs
1676	Table 1	Final evaluator comparison	Seven methods; frozen test groups; independent runs; rankings where available; coverage
1677	Figure 3	Breadth, heterogeneity, deployment trade-off	Admitted task list; paired Seed/GEPA effects; actual per-output costs
1678	Figure 4	Search dynamics and mechanism	Cost-stamped checkpoints; four control conditions; native baselines; reviewed directions
1679	Table 2	Downstream human benefit	New C2 outputs and votes; C3 only after authorized training; regression labels
1680	Table A1	Candidate scope	Fixed task codes and standards; not completion counts
1681	Table A2	Resource roles	Primary source, version, exact assets, feedback construct, permissions
1682	Table A3	Domain admission	Group counts, splits, adapter smoke checks, label availability
1683	Table A4	Feedback route	Direct/derived/new origin and budget definition
1684	Table A5	Label quality	Individual votes, abstentions, rater groups, timing
1685	Table A6	Leakage controls	Access logs, grouping and reserved-metric audit
1686	Table A7	Baseline fidelity	Official code revisions, wrappers, editable scope, allowed evidence
1687	Table A8	Practical cost	All search/diagnostic/failed calls, deployed latency and cost
1688	Figure A1	Evaluation program interfaces	Existing conceptual runtime diagram; no empirical fill needed
1689	Table A9	Full C1 breakdown	Domain-by-method-by-run results, dimensional endpoints, denominators
1690	Table A10	Controlled questions	Implemented intervention flags and comparison prerequisites
1691	Table A11	Search-control effects	Four matched controls, same-space contrast, independent runs
1692	Figure A2	Feedback efficiency and interaction	Nested label budgets, measured annotation time, fresh seed-based searches
1693	Table A12	Mining / end-to-end estimates	Sampling policies and two-by-two cells at each fixed budget
1694	Table A13	Evidence and feedback attribution	Paired compatible interventions and same-group label-detail runs
1695	Figure A3	What changed within a direction?	Candidate code hashes, parent links, observations, diagnoses, outcomes, costs
1696	Table A14	Downstream controls and failures	Per-arm outputs, new blind votes, no-critic control, validity
1697	Table A15	Protected attributes	Reserved metric implementations and human dimensional outcomes
1698	Table A16	Conditional C3	Frozen training manifest, preflight gates, checkpoints, new outcomes
1699	Table A17	Scoped extensions	Separate authorization/readiness and matched controls
1700	Tables A18, A19	Completion and reproduction	Display status plus complete run manifests and artifact hashes

1722 N REPRODUCIBILITY MANIFEST AND DRAFT DATA

1724 Table A19 lists the unresolved values that must be frozen before confirmatory runs. A structural template is not a completed experiment. The active draft uses the author-requested aggregate scenario in `data/ideal_scenario/scenario.json`. One renderer generates its tables, numerical prose macros, and figures, retaining provenance and file hashes in `data/ideal_scenario/manifest.json`. These assignments fix the reporting layout and illustrate a favorable pattern;

they are neither predictions nor outputs from model runs, training, or human annotation. No raw observations, run-level variation, or confidence intervals are generated. Captions and the title-page notice disclose this status without adding banners inside the plots. The empty schemas in `data/experiment_templates/` remain reserved for actual observations. Real-data replacement requires those records, a frozen manifest, recomputed uncertainty, and a renewed claim audit. The paper repository also contains historical simulated plotting fixtures under `data/simulated/` and `figures/simulated/`, generated by `scripts/render_simulated_results.py` and the landscape scripts. They remain available as visual references with their original simulation disclosures and hashes. They are no longer used as the current results section or empirical appendix. Their old source-shift labels and assigned costs must not be relabeled as measured within-domain results or deployment costs.

Table A19: Run manifest to complete before execution. Unset fields are intentional and must not be inferred from historical toy fixtures or unrelated implementation defaults.

Field	Required value
Domain and data	Standard, dataset revision, source/license, duplicate groups, input/output hashes, admitted study roles: <i>TBD</i>
Human protocol	Rubric, dimensions, label origin, raters, consent, compensation, ties, adjudication, timing: <i>TBD</i>
Partitions	Original-input/source split manifests, sizes, label-access log, independent final cohort: <i>TBD</i>
Mining	Signal versions, source groups, margins, random anchor, qualification, budgets, policy seeds: <i>TBD</i>
Models and tools	Evaluator, optimizer, tool-model and generator checkpoints; decoding and API versions: <i>TBD</i>
Baseline adapters	Official code commits, native settings, wrapper changes, shared interfaces, smoke tests: <i>TBD</i>
Search	Seed program, editable space, acceptance checks, native end/parent rules, cost cap and checkpoints: <i>TBD</i>
Measurement	<i>R</i> , retries, precision, completeness, frozen cache, independent search runs, failure policy: <i>TBD</i>
Downstream	Fresh output generation, critic controls; conditional C3 preflight, training, checkpoints: <i>TBD</i>
Analysis	Primary contrasts, paired/clustering rule, multiplicity, minimum effect, sample sizes, exclusions: <i>TBD</i>
Artifacts	Run IDs, code/data hashes, raw traces, labels, derived endpoints, plotting revision, release permissions: <i>TBD</i>

Release only authorized and appropriately anonymized artifacts. Every populated result must link to immutable source records, its denominator, the analysis version, and the relevant frozen manifest. Before a results-bearing submission, revise the abstract, introduction, discussion, and conclusion against those actual outcomes. No numerical or verbal claim becomes established by removing a placeholder notice.